

Python Stock Data Analysis

파이썬을 활용한 주식 데이터 분석

강의 내용

- 최소한의 Python 기초
- 주식 데이터의 이해
- 주식 데이터 수집 및 전처리
- 주식 데이터 분석 지표 계산
- 주식 데이터 시각화
- 국내 주식 및 재무 데이터 수집
- 퀀트 투자 전략 백테스트

최소한의 Python 기초

Python이란?

- Python은 네덜란드 출신의 컴퓨터 프로그래머인 귀도 반 로섬^{Guido van Rossum}이 1991년에 발표한 프로그래밍 언어입니다. 귀도는 Google, Dropbox, Microsoft 등에서 활동했습니다.
- Python의 시작은 1989년 12월로 거슬러 올라갑니다. 당시 크리스마스 시즌이라 암스테르담의 연구소 사무실이 문을 닫자, 귀도는 집에 있는 컴퓨터로 이전부터 구상하던 새로운 스크립트 언어의 인터프리터를 작성하기 시작했고, 이것이 훗날 Python으로 발전했습니다.
- Python이라는 이름은 당시 귀도가 좋아하던 코미디 쇼인 "Monty Python's Flying Circus"에서 따온 것입니다.



- Python은 그리스 신화에 나오는 큰 뱀을 뜻하는 단어입니다.
- Python 버전은 2.x와 3.x가 있습니다. # [참고] Python 2.x는 공식 지원이 종료된 구버전이며, 데이터 분석 실무와 학습에서는 Python 3.x를 사용합니다.

Python의 인기

- PYPL 지수 Popularity of Programming Language Index 기준
현재 가장 인기 있는 언어는 Python 입니다.
- PYPL 지수는 구글에서 각 프로그래밍 언어의
튜토리얼이 얼마나 자주 검색되는지를 분석
하여 만들어집니다.
 - 매월 전세계 기준 결과를 공개합니다.
- Python은 범용 목적으로 개발된 언어이지만
최근 데이터 분석과 머신러닝, 딥러닝 분야
에서 많이 사용됩니다.

Worldwide, Jun 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	45.23 %	+13.5 %
2		Java	10.57 %	-4.6 %
3	↑	C/C++	10.55 %	+3.6 %
4	↑↑	R	5.01 %	+0.1 %
5	↓↓	JavaScript	4.23 %	-3.5 %
6	↑↑↑↑	Objective-C	3.03 %	+0.6 %
7	↑	Rust	2.43 %	-0.3 %
8	↓↓↓	C#	2.37 %	-3.4 %
9	↓↓	PHP	2.36 %	-1.2 %
10	↑	Swift	2.35 %	+0.2 %

출처: <https://pypl.github.io/PYPL.html>

Python의 특징

- Python은 인간다운 언어이므로 사람이 생각하는 방식으로 프로그래밍합니다.
 - Python은 영어 문장처럼 읽히는 문법을 지향합니다.
- Python은 문법 구조가 단순하여 상대적으로 진입 장벽이 낮은 언어입니다.
 - 프로그래밍 사고에 익숙하지 않은 초심자는 개념을 이해하는데 일정 시간이 필요합니다.
- Python은 오픈소스이므로 수많은 서드파티 라이브러리를 공짜로 사용할 수 있습니다.
 - 라이브러리 버전이 다르면 에러를 일으키고, 코딩 스타일에서 일관성이 부족합니다.
- Python은 간결한 코딩이 가능하지만, 들여쓰기 규칙을 준수해야 합니다.

출처: 점프 투 파이썬(<https://wikidocs.net/6>)에서 일부 발췌한 것입니다.

[참고] Excel과 Python의 장단점 비교

구분	Excel	Python
장점	- 복잡한 프로그래밍 지식이 필요 없음 - 데이터를 시각적으로 표현하기 쉬움 - 별도의 패키지를 설치하지 않아도 됨 - 실수를 되돌릴 수 있음(undo)	- 대량의 데이터를 효과적으로 처리 가능 - 다양한 데이터 분석 및 머신러닝 가능 - 반복 작업 및 자동화가 쉬움 - 작업 과정을 코드로 남길 수 있어 재현 및 공유하기 쉬움
단점	- 대량의 데이터를 처리하기 어려움 - 복잡한 분석을 수행하는 데 제한이 있음 - 반복적이고 복잡한 작업의 자동화가 어려움 - 공동 작업에서 버전 관리를 하기 어려움	- 프로그래밍 지식이 필요함 - 전체 데이터를 한 눈에 보기 어려움 - 초기 환경 설정이 필요하고 어려울 수 있음 - 코드 실행 결과가 원본 객체에 반영되었다면 undo할 수 없음

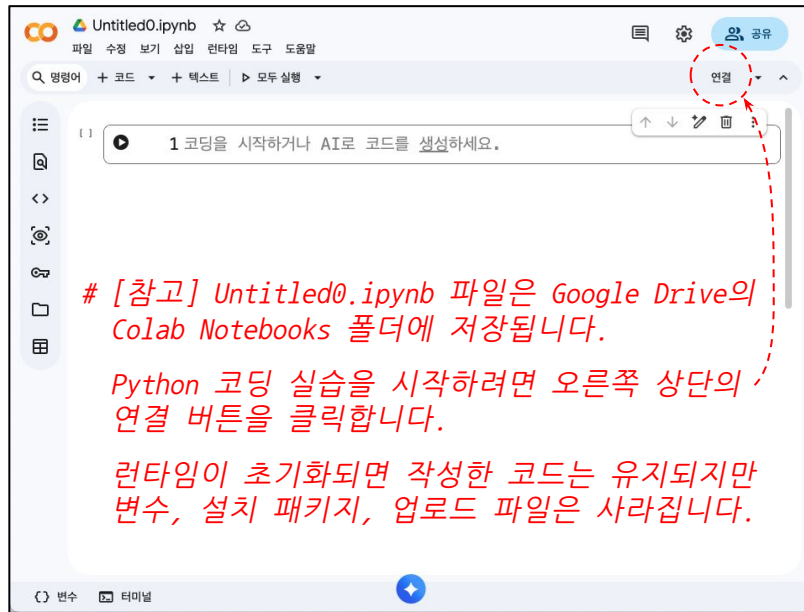
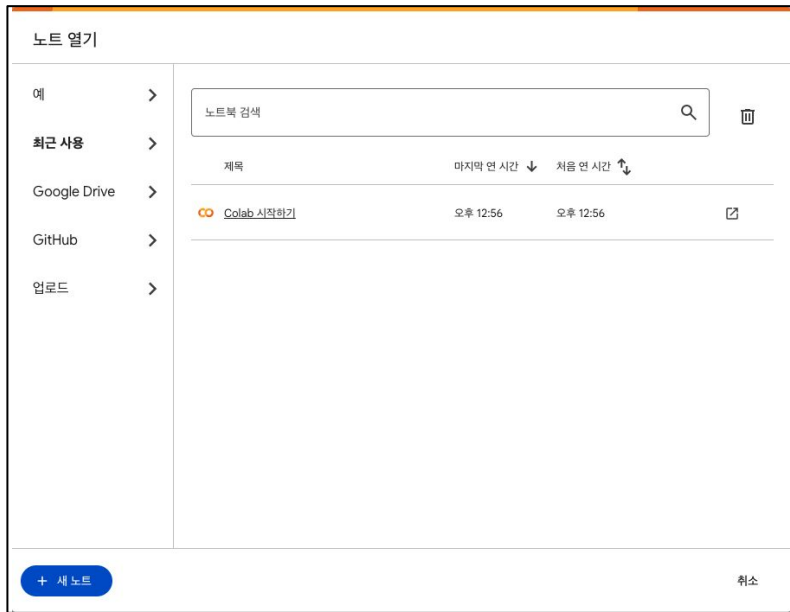
[참고] Python 객체에 어떤 동작을 실행한 결과를 재할당하면 되돌릴 수 없으므로 미리 원본 객체를 복제하는 것이 좋을 수 있습니다.

Colab이란?

- Colab은 Google이 웹 브라우저에서 Python 스크립트를 작성하고 실행할 수 있도록 개발한 온라인 에디터입니다.
 - Colab은 Colaboratory의 줄임말입니다.
- Google 계정이 있으면 Colab을 편리하게 이용할 수 있습니다.
 - Google에 로그인된 상태에서 다른 사람이 만든 Colab 파일을 복사하여 자신의 Google Drive에 사본으로 저장할 수 있습니다.
- 웹 브라우저에서 Colab을 검색하고 'Colab 시작하기'를 클릭하면 관련 메뉴가 열립니다.
 - 링크: <https://colab.research.google.com/?hl=ko>

[참고] Colab 새 노트북 파일 열기

- 왼쪽 하단의 **+ 새 노트** 버튼을 클릭하면, 새 탭에서 Jupyter Notebook이 열립니다.



Colab에서 Python 코딩 실습

- Colab에서 코드 또는 텍스트를 입력하는 부분을 셀^{cell}이라고 합니다.
- 셀의 위/아래에 마우스를 가져다 놓으면 **+ 코드**와 **+ 텍스트** 버튼이 나타납니다.
 - + 코드** 버튼을 클릭하면 Python Code를 입력할 수 있는 셀을 추가합니다.
 - + 텍스트** 버튼을 클릭하면 Markdown, HTML tag 등 웹 문서 작성 셀을 추가합니다.



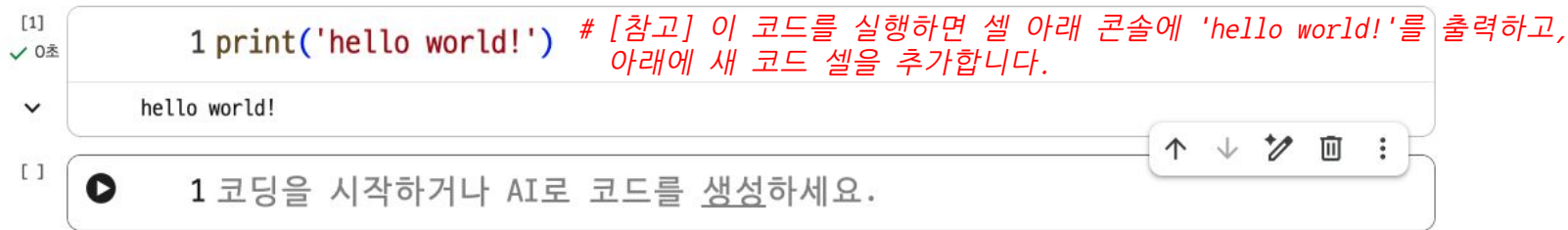
코드 실행 및 셀 추가 실습

- 현재 셀에 다음 코드를 입력합니다.

```
>>> print('hello world!')
```

[참고] 해시(#)로 시작하는 문자열은 주석으로 간주하여 실행하지 않습니다.
주석은 일반적으로 코드 위 또는 오른쪽에 작성합니다.

- Python 코드 셀을 실행하는 두 가지 방법 중 하나를 선택하여 셀을 실행합니다.
 - 코드 셀 왼쪽에 있는 삼각형(▶) 버튼을 클릭합니다.
 - 코드 셀에서 shift + enter 키를 동시에 누릅니다.



셀 메뉴 소개

- 셀 오른쪽 상단에 있는 메뉴를 활용하면 셀에 대한 다양한 동작을 수행할 수 있습니다.

구분	상세 내용
셀 이동	위쪽 화살표 또는 아래쪽 화살표를 클릭합니다.
셀 삭제	휴지통을 클릭합니다.
실행취소	수정 메뉴의 실행취소를 선택합니다.
셀 선택, 복사, 잘라내기	삼점(:) 메뉴 버튼을 클릭하고 하위 메뉴를 선택합니다.
셀 붙여넣기	ctrl + v 키를 동시에 누르면 커서 아래에 셀을 붙여넣습니다.

[]



1 코딩을 시작하거나 AI로 코드를 생성하세요.

+ 코드

+ 텍스트



작업 경로 확인 및 변경

- 필요한 모듈을 임포트합니다.

```
>>> import os
```

- 현재 작업 경로를 확인합니다.

```
>>> os.getcwd() # [참고] Colab에서 기본 작업 경로는 /content 폴더입니다.
```

- sample_data 폴더로 작업 경로를 변경합니다.

```
>>> os.chdir(path='/content/sample_data') # [참고] sample_data 폴더는 현재 작업 경로 안에 있습니다.
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> sorted(os.listdir()) # [참고] sample_data 폴더에는 Colab에서 기본으로 제공하는 여섯 개의 예제 파일이  
저장되어 있으며, 파일 구성은 Colab 환경에 따라 달라질 수 있습니다.
```

구글 드라이브 마운트

- 필요한 모듈을 임포트합니다.

```
>>> from google.colab import drive
```

- 구글 드라이브에 연결합니다.

```
>>> drive.mount(mountpoint='/content/drive')
```

- 오른쪽 팝업이 열리면 'Google Drive에 연결' 버튼을 클릭합니다.
 - 구글 계정을 선택하고, 구글 드라이브에 연결할 항목을 선택합니다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

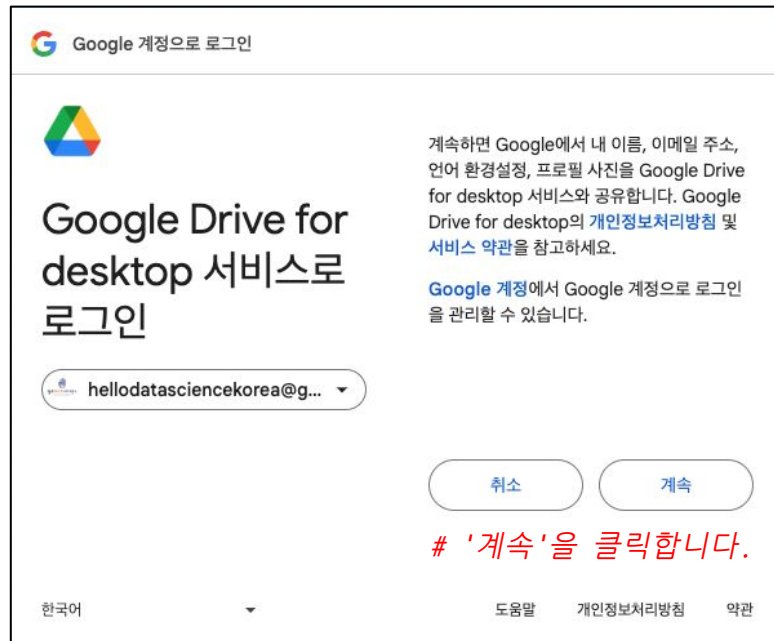
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요

Google Drive에 연결

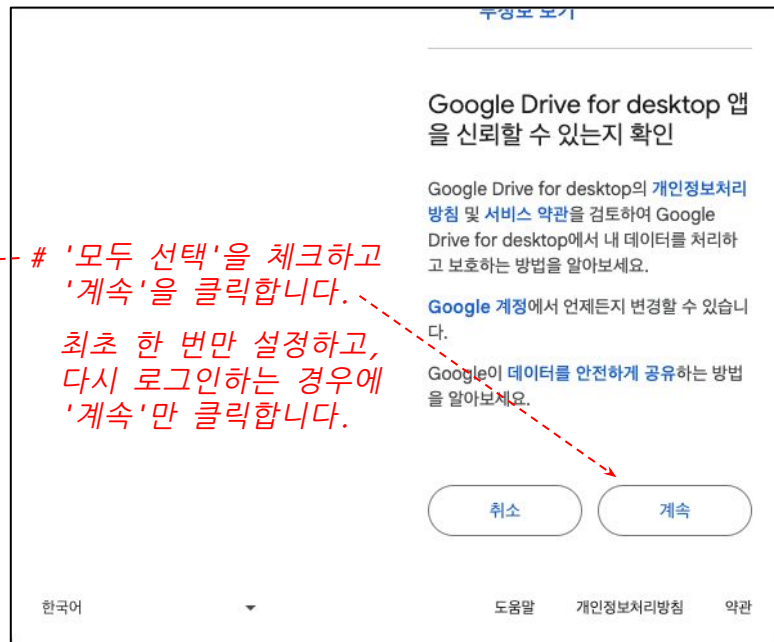
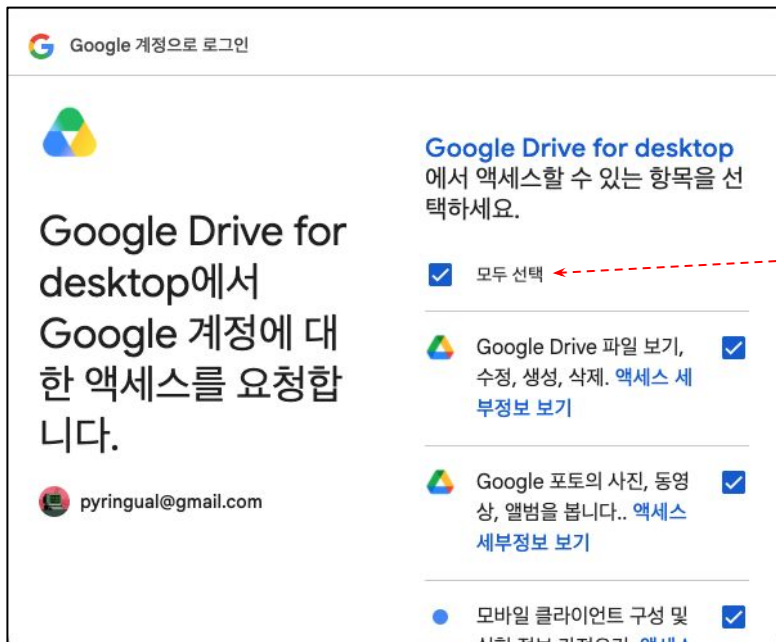
구글 드라이브 마운트(계속)

- 구글 드라이브에 연결할 계정을 선택하고, 로그인합니다.



구글 드라이브 마운트(계속)

- Google 계정에 대한 액세스 요청 항목을 선택합니다.



구글 드라이브에 새 폴더 생성

- 새 폴더명을 path에 할당합니다.

```
>>> path = '/content/drive/MyDrive/Quant'
```

- 구글 드라이브의 내 드라이브 폴더에 새 폴더를 생성합니다.

```
>>> os.makedirs(name=path, exist_ok=True)
```

[참고] Downloads 폴더에서 압축 해제한 파일을 구글 드라이브 workspace 폴더에 업로드한 후, 마지막 코드를 실행하면 실습 파일명 목록을 확인할 수 있습니다.

- 새로 생성한 폴더로 작업 경로를 변경합니다.

```
>>> os.chdir(path=path)
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> sorted(os.listdir())
```

[참고] 구글 드라이브에 실습 파일을 저장하면 런타임이 초기화되어도 파일을 계속 사용할 수 있습니다. 단, 사용자가 직접 삭제하거나 저장 공간이 변경되면 파일이 사라질 수 있습니다.

[참고] Python 기초 문법

- 데이터 분석을 위한 Python 기초 문법은 다음 내용을 주로 다룹니다.
 - 변수와 객체: 객체 지향 프로그래밍의 특징
 - 단순 자료형: 정수, 실수, 문자열, 부울
 - 복합 자료형: 리스트, 튜플, 집합, 딕셔너리
 - 조건문: if, 삼항 연산자
 - 반복문: for, while, 리스트 컴프리헨션
 - 함수: def, lambda
 - 클래스: class

주식 데이터의 이해

주식 데이터의 구조

- 주식 데이터는 시간의 흐름에 따라 기록되는 대표적인 시계열 데이터입니다.
 - 각 시점마다 주식의 가격과 거래량과 같은 정보가 함께 기록됩니다.
- 주식 데이터는 날짜시간 인덱스와 여러 수치형 변수(OHLCV)로 구성된 데이터프레임의 형태로 표현됩니다.
 - 주식 데이터는 날짜를 기준으로 정렬되며, 각 날짜마다 동일한 구조를 갖습니다.
 - 다섯 가지 수치형 변수는 주식 가격 움직임과 거래 강도를 설명하는 핵심 지표입니다.

Datetime	Open	High	Low	Close	Volume
거래일시	시가	최고가	최저가	종가	거래량

[참고] 시계열 데이터 vs 횡단면 데이터

- 시계열 데이터란 시간의 흐름에 따라 순차적으로 관측한 데이터입니다.
 - 개별 관측값은 특정 시점에 기록되며, 관측값 사이의 순서는 단순한 정렬 정보가 아니라 데이터가 어떤 현상을 설명하는지를 결정하는 핵심 요소입니다.
 - 대표적인 시계열 데이터는 주가, 매출, 기온, 방문자 수 등이 있습니다.
- 횡단면 데이터는 동일한 시점에서 여러 개체를 동시에 관측한 데이터입니다.
 - 횡단면 데이터는 샘플의 순서를 섞어도 분석 결과에 큰 영향을 주지 않는 경우가 많지만 시계열 데이터에서는 이러한 접근이 허용되지 않습니다.
 - 시계열 데이터의 관측값 순서를 변경하면 원래의 현상을 반영하지 못하게 됩니다.

[참고] 시계열 데이터 vs 횡단면 데이터

	대여량	기온(℃)	습도(%)	요일
2020-01-01	128	3.2	72	수요일
2020-01-02	121	4.8	67	목요일
2020-01-03	157	6.4	58	금요일
2020-01-04	194	8.2	52	토요일
2020-01-05	255	9.6	47	일요일
⋮	⋮	⋮	⋮	⋮

시계열 데이터

[참고] 시계열 데이터는 시간의 흐름에 따라 순차적으로 관측한 데이터입니다.

횡단면 데이터

[참고] 횡단면 데이터는 동일한 시점에서 여러 개체를 동시에 관측한 데이터입니다.

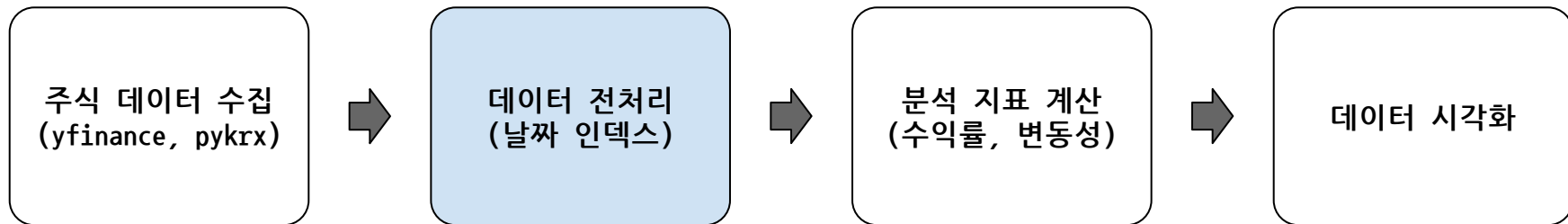
주식 데이터의 구성 요소

- 주식 데이터는 다섯 가지 핵심 변수로 구성됩니다.
 - Open(시가): 장이 시작될 때 형성된 첫 거래 가격입니다. # [참고] 시가는 장 시작 전 동시호가로 결정되는 가격입니다.
 - High(고가): 해당 날짜 동안 거래된 가격 중 가장 높은 값입니다.
 - Low(저가): 해당 날짜 동안 거래된 가격 중 가장 낮은 값입니다.
 - Close(종가): 장이 종료될 때 마지막으로 거래된 가격입니다.
 - 종가는 시장 참여자들의 합의된 최종 가격이므로 데이터 분석에서 가장 많이 사용됩니다.
 - Volume(거래량): 해당 날짜 동안 거래된 주식의 총 수량입니다.
 - 거래량은 가격 움직임의 신뢰도를 판단하는 기준으로 활용됩니다.

날짜 인덱스의 중요성

- 주식 데이터에서 인덱스는 단순한 번호가 아니라 시간 정보를 의미합니다.
 - 특정 기간을 선택하거나 이동 평균을 계산할 때 날짜 인덱스가 중요한 역할을 합니다.
- 주식 데이터를 다음 과정을 통해 전처리합니다.
 - 거래일을 인덱스로 설정합니다.
 - 날짜 인덱스를 기준으로 데이터를 정렬합니다.
- 데이터프레임에서 날짜시간형 시리즈를 인덱스로 설정하면, `DatetimeIndex`로 변환됩니다.
 - `DatetimeIndex`는 날짜 요소 추출, 선형 보간법 등을 쉽게 처리하는 장점이 있습니다.

주식 데이터 분석 프로세스



주식 데이터 수집 및 전처리

yfinance 라이브러리 소개

- yfinance는 Yahoo Finance의 공개 데이터를 기반으로 주식, ETF, 지수 등의 시계열 데이터를 손쉽게 수집할 수 있도록 지원하는 Python 라이브러리입니다.
 - 주식 가격 데이터(Open, High, Low, Close, Volume)와 일부 재무 데이터를 데이터프레임 형태로 불러올 수 있어, 데이터 분석에서 널리 활용됩니다.
 - 별도의 인증키 없이도 글로벌 주식 데이터를 즉시 수집할 수 있다는 장점이 있습니다.
 - 웹 데이터를 기반으로 제공하는 서비스이므로, 데이터의 안정성과 지속성이 보장되지 않으며 호출 제한이나 구조 변경에 영향을 받을 수 있습니다.
 - 미국 주식 데이터의 경우 안정성과 정확도가 높은 편이지만, 국내 주식 데이터의 경우 데이터 누락, 수정 종가 미반영, 가격 불일치 등의 문제가 발생할 수 있습니다.

[참고] 수정 종가

- 배당, 액면분할, 무상증자 등의 기업 이벤트가 발생하면 주가가 변동됩니다.

구분	상세 내용
배당	배당락일에 주가가 배당금만큼 하락합니다.
액면분할/무상증자	주식 수가 증가하므로 주가는 액면분할 비율만큼 하락합니다.

- 수정 종가는 기업 이벤트의 영향을 반영하여 과거 가격을 현재 기준으로 보정한 가격입니다.
 - 장기 수익률 분석이나 백테스트에서는 수정 종가를 사용하는 것이 일반적입니다.
 - 국내 데이터는 수정 종가를 제공하지 않는 경우가 많으므로, 배당 및 액면분할 정보를 이용하여 직접 계산해야 합니다.

[참고] 이번 실습에서는 국내 주식 데이터의 한계 때문에 완전한 수정 종가 기반의 백테스트를 진행하지 않습니다.

필요한 모듈 импорт

- Colab에서 필요한 라이브러리 설치 여부를 확인합니다.

```
>>> !pip list | grep yfinance
```

[참고] 코드 실행 결과 콘솔에 아무것도 출력되지 않으면 현재 Colab에는 'yfinance'를 포함하는 라이브러리가 설치되지 않았다는 것을 의미합니다.

- 필요한 라이브러리를 설치합니다.

```
>>> !pip install yfinance
```

- 필요한 모듈을 импорт합니다.

```
>>> import numpy as np
```

```
>>> import pandas as pd
```

```
>>> import yfinance as yf
```

주식 데이터 수집

- 티커를 설정합니다.

```
>>> ticker = '005930.KS'
```

- 주식 데이터 수집 시작일자를 설정합니다.

```
>>> start_date = '2019-12-31' # [참고] 단순 수익률을 계산할 때 첫 번째 행은 결측이 되므로 시작일자를  
# 데이터 수집 기간의 시작일보다 하루 앞당긴 날짜로 지정합니다.
```

- 주식 데이터 수집 종료일자를 설정합니다. # [참고] 주가 수집 함수가 마지막 일자를 포함하지 않습니다.
아래 코드를 실행하면 시스템 날짜를 end_date에 할당합니다.

```
>>> end_date = pd.Timestamp.today().strftime(format='%Y-%m-%d')
```

- 주식 데이터를 수집하고 데이터프레임을 생성합니다.

```
>>> df = yf.download(tickers=ticker, start=start_date, end=end_date)
```

데이터프레임 구조 확인

- df의 처음 5행을 확인합니다.

```
>>> df.head() # [참고] n 매개변수에 출력할 행 개수를 정수로 지정합니다.(기본값: 5)
```

- df의 열 이름을 확인합니다.

```
>>> df.columns # [참고] 열 이름이 멀티 인덱스로 생성되었습니다.  
열 이름 네임은 ['Price', 'Ticker']로 설정되었습니다.
```

- df의 열 이름에서 첫 번째 레벨만 남깁니다.

```
>>> df.columns = df.columns.get_level_values(level=0) # [참고] 열 이름의 1번 레벨을 삭제합니다.  
>>> df.columns.droplevel(level=1)
```

- df의 열 이름 네임을 삭제합니다.

```
>>> df.columns.name = None # [참고] df의 열 이름 네임이 'Price'로 남아 있습니다.  
왼쪽 코드를 실행하면 열 이름 네임을 초기화합니다.
```

주식 데이터 다시 수집

- 주식 데이터를 수집하고 데이터프레임을 생성합니다.

```
>>> df = yf.download(tickers=ticker,  
  
                      start=start_date,  
  
                      end=end_date,  
  
                      auto_adjust=False, # [참고] auto_adjust 매개변수에 False를 지정하면 OHLC를 구조를  
                                         유지하고, 수정 종가를 추가합니다.(기본값: True)  
  
                      progress=False, # [참고] progress 매개변수에 False를 지정하면 다운로드 진행 표시를  
                                       출력하지 않습니다.(기본값: True)  
  
                      multi_level_index=False) # [참고] multi_level_index 매개변수에 False를 지정하면  
                                                단일 레벨 열 이름을 반환합니다.(기본값: True)
```


데이터프레임 전처리

- df의 처음 5행을 확인합니다.

```
>>> df.head() # [참고] df의 첫 번째 열은 Adj Close(수정 종가)입니다.
```

- df의 열 순서를 변경하기 위해 열 이름 리스트를 생성합니다.

```
>>> cols = ['Open', 'High', 'Low', 'Close', 'Volume']
```

- df의 열 순서를 변경하고 df에 재할당합니다.

```
>>> df = df.loc[:, cols]
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

날짜 인덱스 전처리

- df의 정보를 확인합니다.

```
>>> df.info() # df의 정보를 확인합니다.  
[참고] 행 개수, 열 개수, 열 이름, 열별 결측값 아닌 개수 및 열별 자료형을 출력합니다.
```

- df의 인덱스를 확인합니다.

```
>>> df.index # [참고] df의 인덱스는 DatetimeIndex입니다.
```

- df의 인덱스를 초기화하고 기존 인덱스를 첫 번째 열로 삽입한 결과를 df에 재할당합니다.

```
>>> df = df.reset_index() # [참고] drop 매개변수에 True를 지정하면 기존 인덱스를 삭제합니다.  
(기본값: False)
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

날짜 인덱스 전처리(계속)

- df에서 Date를 문자열로 변환합니다.

```
>>> df['Date'] = df['Date'].astype(dtype=str)
```

- df의 열별 자료형을 확인합니다.

```
>>> df.dtypes # [참고] Date의 자료형이 object입니다.
```

- df의 행 순서를 무작위로 섞은 결과를 df에 재할당합니다.

```
>>> df = df.sample(frac=1, random_state=1, ignore_index=True)
```

- df의 처음 5행을 확인합니다.

```
>>> df.head() # [참고] df의 인덱스는 정수 0부터 시작하지만 Date가 무작위로 섞여 있습니다.
```

날짜 인덱스 전처리(계속)

- df에서 Date를 날짜시간형으로 변환합니다.

```
>>> df['Date'] = pd.to_datetime(arg=df['Date'])
```

- df에서 Date를 인덱스로 설정합니다.

```
>>> df = df.set_index(keys='Date') # [참고] 시계열 데이터는 시간 순서가 매우 중요하므로 날짜시간  
# 데이터를 인덱스로 설정해야 합니다.
```

- df를 인덱스 기준으로 오름차순 정렬합니다.

```
>>> df = df.sort_index() # [참고] 날짜 인덱스 순으로 정렬해야 단순 수익률 및 이동 통계량을 계산할 수  
# 있습니다.
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

[참고] 누락된 인덱스 추가 및 결측값 처리

- df의 인덱스를 일(day) 단위로 재생성하고 처음 5행만 선택하여 df_na에 할당합니다.

```
>>> df_na = df.asfreq(freq='D').head()
```

- df_na를 확인합니다.

```
>>> df_na
```

- df_na에서 종가에 있는 결측값을 직전 값으로 채운 결과를 확인합니다.

```
>>> df_na['Close'].ffill()
```

- df_na에서 종가에 있는 결측값을 선형 보간법으로 채운 결과를 확인합니다.

```
>>> df_na['Close'].interpolate(method='linear') # [주의] 주식 데이터의 결측값을 선형 보간법으로  
# 함부로 채우면 안 됩니다.
```

[참고] 전역 변수 목록 확인 및 변수 삭제

- 전역 변수 목록을 확인합니다.

```
>>> %whos
```

- 전역 변수 목록에서 데이터프레임만 확인합니다.

```
>>> %whos DataFrame
```

- df_na를 전역 변수 목록에서 삭제합니다.

```
>>> del df_na
```

- 전역 변수 목록에서 데이터프레임만 확인합니다.

```
>>> %whos DataFrame
```

날짜 인덱스 필터링

- df에서 2020년도 데이터만 선택합니다.(인덱싱)

```
>>> df.loc['2020'] # [참고] df에는 'yyyy' 형태의 행 이름은 없지만, df의 인덱스가 날짜 인덱스이므로  
네자리 정수를 지정하여 원하는 연도를 인덱싱할 수 있습니다.
```

- df에서 2020년 1월 데이터만 선택합니다.(인덱싱)

```
>>> df.loc['2020-01'] # [참고] 'yyyy-mm' 형태의 지정하여 특정 연도와 월까지 함께 지정할 수 있습니다.  
[주의] 인덱스의 부분 문자열에 대해 팬시 인덱싱과 슬라이싱을 지원하지 않습니다.
```

- df에서 2020년 1월 2일과 3일 데이터만 선택합니다.(팬시 인덱싱)

```
>>> df.loc[['2020-01-02', '2020-01-03']]
```

- df에서 2020년 1월 1~7일 데이터만 선택합니다.(슬라이싱)

```
>>> df.loc['2020-01-01':'2020-01-07']
```

[참고] 날짜시간 인덱스의 속성 및 메서드 소개

구분	상세 내용
<code>df.index.year</code>	<code>df.index</code> 의 원소에서 년(year)을 정수형 시리즈로 반환합니다.
<code>df.index.month</code>	<code>df.index</code> 의 원소에서 월(month)을 정수형 시리즈로 반환합니다.
<code>df.index.day</code>	<code>df.index</code> 의 원소에서 일(day)을 정수형 시리즈로 반환합니다.
<code>df.index.hour</code>	<code>df.index</code> 의 원소에서 시(hour)을 정수형 시리즈로 반환합니다.
<code>df.index.minute</code>	<code>df.index</code> 의 원소에서 분(minute)을 정수형 시리즈로 반환합니다.
<code>df.index.second</code>	<code>df.index</code> 의 원소에서 초(second)를 정수형 시리즈로 반환합니다.
<code>df.index.dayofweek</code>	<code>df.index</code> 의 원소에서 요일을 정수형 시리즈로 반환합니다.(Monday: 0, Sunday: 6)
<code>df.index.day_name()</code>	<code>df.index</code> 의 원소에서 요일을 문자열 시리즈로 반환합니다.(로케일에 따라 다름)
<code>df.index.strftime()</code>	<code>df.index</code> 의 원소에서 지정한 날짜시간 포맷의 문자열 시리즈로 반환합니다.

[참고] 날짜시간 주요 포맷

구분	상세 내용	예시	구분	상세 내용	예시
%Y	연도(4자리)	'2025'	%p	AM/PM 표기	'AM'
%y	연도(2자리)	'25'	%M	분(정수, 00~59)	'02'
%m	월(정수, 01~12)	'12'	%S	초(정수, 00~59)	'03'
%B	월(Locale 전체)	'December'	%f	백만 분의 1초	'456789'
%b	월(Locale 단축)	'Dec'	%s	유닉스 시간 ^{UNIX time}	'1703952123'
%d	일(정수, 01~31)	'31'	%W	연중 주의 순번(00~52)	'52'
%j	연중 일의 순번(001~366)	'365'	%w	주중 요일의 순번(0~6)	'0'
%H	시(24시간, 00~23)	'01'	%A	요일(Locale 전체)	'Sunday'
%I	시(12시간, 01~12)	'01'	%a	요일(Locale 단축)	'Sun'

[참고] DatetimeIndex의 Frequency 문자열

구분	상세 내용
B	영업일(월요일~금요일) 단위
D	일 단위
W	주 단위(일요일 기준 주간 집계)
ME, MS	매월 말일, 매월 시작일
BME, BMS	영업일 기준 매월 말일, 매월 시작일
QE, QS	분기 말일, 분기 시작일
BQE, BQS	영업일 기준 분기 말일, 분기 시작일
YE, YS	연도 말일, 연도 시작일
BYE, BYS	영업일 기준 연도 말일, 연도 시작일

구분	상세 내용
W-MON	매주 월요일(요일 앵커 사용)
YE-JAN	매년 1월 말일(월 앵커 사용)
h	매시 단위
bh	영업일 기준 매시 단위(09~16시)
min	매분 단위
s	매초 단위
ms	천 분의 1초 단위
us	백만 분의 1초 단위
ns	십억 분의 1초 단위

날짜 인덱스의 속성 활용

- df에서 매년 1월 데이터만 선택합니다.(불리언 인덱싱)

```
>>> df.loc[df.index.month == 1]
```

- df에서 매년 12월 31일 데이터만 선택합니다.

```
>>> df.loc[(df.index.month == 12) & (df.index.day == 31)]
```

[참고] 비트 연산자가 비교 연산자보다 우선순위가 높으므로 비교 연산 코드를 소괄호를 묶어주어야 합니다.

- df의 인덱스에서 정수 요일을 생성합니다.

```
>>> df.index.dayofweek # [참고] 정수 0은 Monday, 정수 6은 Sunday입니다.
```

- df에서 매주 월요일 데이터만 선택합니다.

```
>>> df.loc[df.index.dayofweek == 0]
```

날짜 인덱스의 속성 활용(계속)

- df의 인덱스에서 영문 요일 문자열을 생성합니다.

```
>>> df.index.strftime(date_format='%A') # [참고] date_format 매개변수에 지정한 날짜 포맷에 맞는  
문자열을 반환합니다.
```

- df의 인덱스에서 영문 요일 문자열을 생성합니다.

```
>>> df.index.day_name() # [참고] locale 매개변수에 'ko_KR'을 지정하면 요일을 한글 문자열로 반환합니다.  
[주의] Colab 환경에서는 한국 로케일을 바로 사용할 수 없습니다.
```

- 한글 요일 문자열을 매핑할 딕셔너리를 생성합니다.

```
>>> weekday_kor = {0: '월', 1: '화', 2: '수', 3: '목', 4: '금', 5: '토', 6: '일'}
```

- df의 인덱스에서 정수 요일을 추출하고, 한글 요일명으로 매핑합니다.

```
>>> df.index.dayofweek.map(mapper=weekday_kor)
```

[참고] 집계 함수 활용법

- df에서 매년 첫 번째 거래일 데이터만 선택합니다.

```
>>> df.groupby(by=df.index.year).first() # [참고] first 메서드는 그룹 집계 함수이므로, 그룹 키를  
# 새로운 인덱스로 설정합니다.
```

- df에서 매년 마지막 거래일 데이터만 선택합니다.

```
>>> df.groupby(by=df.index.year).last()
```

- df에서 매년 첫 번째 거래일 데이터만 선택합니다.

```
>>> df.groupby(by=df.index.year).nth(n=0) # [참고] nth 메서드는 n번째 행을 선택하는 함수이므로,  
# 원본 인덱스를 유지합니다.
```

- df에서 매년 마지막 거래일 데이터만 선택합니다.

```
>>> df.groupby(by=df.index.year).nth(n=-1)
```

주식 데이터 분석 지표 계산

수익률, 변동성, 이동 평균

주식 데이터 분석 지표

구분	지표	정의	특징
수익률	단순 수익률	전일 대비 종가 변화율	단순 수익률은 가격의 상대적 변화를 직관적으로 나타내며, 가장 기본적인 성과 측정 지표입니다.
수익률	로그 수익률	종가 비율의 자연로그 값	로그 수익률은 시계열 모델링과 통계 분석에 적합하며, 수학적으로 다루기 쉬운 수익률 지표입니다.
변동성	이동 표준편차	일정 기간 수익률의 표준편차	변동성은 가격의 불확실성과 리스크 수준을 나타내며, 수익률의 분산 정도를 기반으로 계산됩니다.
추세	이동 평균	일정 기간 종가의 평균	이동 평균은 가격의 잡음을 제거하고, 상승/하락 추세를 직관적으로 보여줍니다.
추세	모멘텀	과거 대비 가격 변화량 또는 변화율	모멘텀은 가격이 상승하거나 하락하는 힘의 방향 및 강도를 나타냅니다.(투자 의사결정 활용 지표)
거래량	거래량 이동 평균	일정 기간 거래량의 평균	거래량은 가격 움직임의 신뢰도를 판단하고, 시장 참여 강도를 나타내는 보조 지표입니다.

[참고] 단순 수익률 vs 로그 수익률

구분	단순 수익률	로그 수익률
정의	전일 대비 종가 변화율($-1 \sim \infty$)	종가 비율에 로그를 취한 값($-\infty \sim \infty$)
수식	$R_t = \frac{P_t}{P_{t-1}} - 1$	$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$
직관성	매우 직관적	직관성은 다소 떨어짐
해석	전일 대비 가격이 몇 % 올랐는가	연속 복리 기준 변화량
누적 수익률	곱셈 필요	덧셈으로 충분
누적 수익률 수식	$R_{\text{cum}} = \prod_{t=1}^T (1 + R_t) - 1$	$R_{\text{cum}} = \exp \left(\sum_{t=1}^T r_t \right) - 1$
활용 분야	실무 보고, 투자 성과 설명	금융공학, 시계열 모형, 수리적 분석

[참고] 대부분의 포트폴리오 이론은 수익률이 정규 분포를 따른다는 가정을 기초로 합니다. 로그 수익률이 정규분포하면 주가가 로그-정규분포를 따른다고 볼 수 있어, 수학적 모델링이 쉬워집니다.

일간 수익률 계산

- 단순 수익률을 계산하고 df에 추가합니다.

```
>>> df['Daily_Return'] = df['Close'].pct_change() # [참고] pct_change 메서드는 직전 값 대비 현재  
# 값의 변화율을 반환합니다.
```

- 로그 수익률을 계산하고 df에 추가합니다.

```
>>> df['Log_Return'] = np.log(df['Close'] / df['Close'].shift(periods=1))
```

- df에서 열별 결측값 개수를 확인합니다.

[참고] shift 메서드는 periods 매개변수에 지정한
정수만큼 위(음수), 아래(양수)로 이동시킵니다.

```
>>> df.isna().sum()
```

- df에서 결측값을 0으로 대체하고 df에 재할당합니다.

```
>>> df = df.fillna(value=0) # [참고] dropna 메서드의 axis 매개변수에 0을 지정하면 결측을 포함하는 행,  
# 1을 지정하면 결측을 포함하는 열을 삭제합니다.(기본값: 0)
```

누적 수익률 계산

- 누적 수익률을 계산하고 df에 추가합니다.

```
>>> df['Cum_Return'] = (df['Daily_Return'] + 1).cumprod() # [참고] 단순 수익률에 1을 더하여  
원금을 1로 설정합니다.
```

- df의 처음 10행을 확인합니다.

```
>>> df.head(n=10)
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```

- 로그 누적 수익률을 계산하고 일반 수익률로 변환합니다.

```
>>> np.exp(df['Log_Return'].cumsum()) # [참고] 로그 누적 수익률을 지수 변환하면 단순 누적 수익률과  
동일한 값이 됩니다.
```

누적 수익률 백분율 계산

- 누적 수익률의 백분율을 계산하고 df에 추가합니다.

```
>>> df['Cum_Return_Pct'] = (df['Cum_Return'] - 1) * 100
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```

- 시작 종가와 마지막 종가를 각각 생성합니다.

```
>>> base_close, last_close = df['Close'].iloc[[0, -1]]
```

- 전체 기간 누적 수익률을 확인합니다.

```
>>> (last_close / base_close - 1) * 100
```

변동성의 이해

- 변동성은 수익률의 얼마나 크게 변하는 지를 나타내는 지표입니다.
 - 주가가 얼마나 크게 오르고 내리는지를 수치로 표현한 것입니다.
- 변동성이 크다는 것은 가격 변동 폭이 커서 투자 위험이 크다는 것을 의미합니다.
 - 반대로 변동성이 작으면 가격이 비교적 안정적으로 움직인다고 볼 수 있습니다.
- 변동성은 일반적으로 수익률의 표준편차로 계산합니다.
 - 수익률이 평균을 중심으로 얼마나 퍼져 있는지를 측정하는 방식입니다.
 - 변동성은 가격 움직임의 강도(크기)를 나타내는 지표라고 이해할 수 있습니다.

변동성의 한계와 동적 변동성

- 전체 기간의 수익률을 기준으로 계산하는 변동성은 하나의 평균적인 값입니다.
 - 특정 시점에서 발생한 시장 변화나 이벤트를 충분히 반영하지 못하는 한계가 있습니다.
 - 금융 시장에서는 변동성이 시간에 따라 지속적으로 변화하는 특징을 보입니다.
- 변동성의 한계를 보완하기 위해 이동 변동성을 사용합니다.
 - 일정 기간 동안의 수익률을 기준으로 변동성을 계산하는 방식입니다.
 - 이동 변동성은 최근 시장 상태를 반영할 수 있으며, 시장이 안정적인지 혹은 불안정한지 여부를 시계열로 확인할 수 있습니다.
 - 일반적으로 20 거래일(약 1개월) 기준의 이동 표준편차를 많이 사용합니다.

변동성 계산

- 단순 수익률의 평균을 확인합니다.

```
>>> df['Daily_Return'].mean()
```

- 단순 수익률의 표준편차를 확인합니다.

```
>>> df['Daily_Return'].std()
```

- 단순 수익률의 20일 이동 표준편차를 계산하고 df에 추가합니다.

```
>>> df['Volatility_20'] = df['Daily_Return'].rolling(window=20).std()
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```

이동 평균

- 이동 평균은 일정 기간 동안의 가격 평균이며, 가격의 흐름을 파악하는 지표입니다.
 - 이동 평균은 가격 데이터를 부드럽게 만들어 전반적인 추세를 파악하는 데 사용됩니다.
- 이동 평균을 활용하면 다음과 같이 해석할 수 있고, 매수/매도 규칙을 만들 수 있습니다.
 - 단기(20일) 이동 평균: 최근 가격 흐름을 반영하며, 가격 변화에 빠르게 반응합니다.
 - 장기(60일) 이동 평균: 전체적인 추세를 반영하며, 부드러운 흐름을 보입니다.

구분	해석
단기 이동 평균 > 장기 이동 평균	상승 추세 → 매수
단기 이동 평균 < 장기 이동 평균	하락 추세 → 매도

[주의] 실제 투자에서는 거래량과 변동성 등 추가적인 조건을 함께 고려합니다.

이동 평균 계산

- 종가의 20일 이동 평균을 계산하고 df에 추가합니다.

```
>>> df['Moving_Avg_20'] = df['Close'].rolling(window=20).mean()
```

- 종가의 60일 이동 평균을 계산하고 df에 추가합니다.

```
>>> df['Moving_Avg_60'] = df['Close'].rolling(window=60).mean()
```

- df의 처음 10행을 확인합니다.

```
>>> df.head(n=10)
```

- df의 마지막 10행을 확인합니다.

```
>>> df.tail(n=10)
```


주식 데이터 시각화

Colab에서 한글 폰트 설치

- 현재 Colab에 설치된 폰트 경로 목록을 확인합니다.

```
>>> !fc-list # [참고] fc-list는 리눅스 fontcoding 계열 명령어로, 현재 설치된 폰트 경로 목록을 출력합니다.
```

- 불필요한 경로를 제거하고 전체 폰트 이름만 출력합니다.

```
>>> !fc-list : family # [참고] 콜론은 fontcoding의 쿼리 구문 시작 기호입니다.  
                        콜론 오른쪽에 조건을 추가하지 않았으므로 경로 없이 전체 폰트 이름만 출력합니다.
```

- Colab에서 사용할 나눔 폰트를 설치합니다.

```
>>> !apt-get install -y fonts-nanum # [참고] apt-get은 리눅스 저장소에 등록된 패키지를 의존성까지  
                                    포함하여 설치/업데이트/삭제하는 명령어입니다.
```

- 불필요한 경로를 제거하고 한글 폰트 이름만 중복 없이 오름차순 정렬하여 출력합니다.

```
>>> !fc-list :lang=ko family | sort | uniq # [참고] 콜론 오른쪽에 한글 폰트만 필터링하는 조건과  
                                           중복 제거 및 오름차순 정렬 옵션을 추가합니다.
```

한글 폰트명 탐색

- 필요한 모듈을 임포트합니다.

```
>>> import matplotlib.font_manager as fm # [참고] 컴퓨터에 설치된 폰트 정보를 확인할 때 사용합니다.
```

```
>>> import matplotlib.pyplot as plt
```

```
>>> import seaborn as sns
```

- 현재 사용 중인 컴퓨터에 설치된 전체 폰트 파일명을 리스트로 생성합니다.

```
>>> fontList = fm.findSystemFonts(fonttext='ttf')
```

- fontList의 원소 개수를 확인합니다.

```
>>> len(fontList)
```

한글 폰트명 탐색(계속)

- fontList에서 특정 폰트명을 포함하는 파일명을 선택하여 fontPath에 할당합니다.

```
>>> fontPath = sorted([font for font in fontList if 'Nanum' in font])
```

- fontPath를 확인합니다.

```
>>> fontPath
```

- 반복문으로 컴퓨터에 설치된 폰트명을 리스트로 반환합니다.

```
>>> [fm.FontProperties(fname=font).get_name() for font in fontPath]
```

- matplotlib 라이브러리의 폰트 관리자 객체를 초기화합니다.

```
>>> fm.fontManager.__init__() # [참고] 컴퓨터에 설치된 폰트 파일들을 다시 스캔하여 내부 폰트 캐시를  
                             재구성하여 새로 설치한 한글 폰트를 사용할 수 있게 합니다.
```

그래프 요소 설정

- 한글 폰트와 글자 크기를 설정합니다. # [참고] rc는 runtime configuration를 의미하며 pyplot을 실행하는 환경을 의미합니다.

```
>>> plt.rc(group='font', family='NanumBarunGothic', size=10)
```

- 그래프 크기와 해상도를 설정합니다.

```
>>> plt.rc(group='figure', figsize=(12, 4), dpi=120)
```

- 축에 유니코드 마이너스를 출력하지 않도록 설정합니다.

```
>>> plt.rc(group='axes', unicode_minus=False) # [참고] x축과 y축 눈금에서 음수 앞에 '-' 대신 '␣'를 출력하도록 않도록 설정합니다.
```

- 범례에 채우기 색과 테두리 색을 추가합니다. # [참고] fc 매개변수에 채우기 색을, ec 매개변수에 테두리 색을 지정합니다. '0'은 검정, '1'은 흰색입니다.

```
>>> plt.rc(group='legend', frameon=True, fc='0.9', ec='0.9')
```

종가 추이 선 그래프 시각화

- 종가 추이를 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Close', color='0', lw=1)
```

```
>>> plt.axhline(y=base_close, color='0.5', ls='--', lw=0.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='가격($)')
```

```
>>> plt.title(label=f'{ticker} 종가($)', fontweight='bold')
```

```
>>> plt.show()
```

누적 수익률 선 그래프 시각화

- 누적 수익률 추이를 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Cum_Return_Pct', color='red', lw=1)
```

```
>>> plt.axhline(y=0, color='0.5', ls='--', lw=0.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='백분율(%))')
```

```
>>> plt.title(label=f'{ticker} 누적 수익률 백분율(%)', fontweight='bold')
```

```
>>> plt.show()
```

로그 수익률 히스토그램 시각화

- 로그 수익률의 기술통계량을 확인합니다.

```
>>> df['Log_Return'].describe()
```

- 로그 수익률을 히스토그램으로 시각화합니다. # [참고] 로그 수익률은 정규분포처럼 보이지만 실제로는 양쪽 꼬리가 정규분포보다 두꺼운 특징을 가집니다.

```
>>> sns.histplot(data=df, x='Log_Return', fc='0.8', ec='1',
```

```
                binrange=(-0.15, 0.15), bins=60, stat='density')
```

```
>>> sns.kdeplot(data=df, x='Log_Return', color='red', lw=1.5)
```

```
>>> plt.title(label=f'{ticker} 로그 수익률 히스토그램', fontweight='bold')
```

```
>>> plt.show()
```


로그 수익률 상자 그림 시각화

- 로그 수익률의 분포를 상자 그림으로 시각화합니다.

```
>>> sns.boxplot(data=df, y='Log_Return', color='0.8', width=0.3, fliersize=5,
                flierprops={'markerfacecolor': 'pink', 'markeredgecolor': 'red'})

>>> plt.xlabel(ylabel=f'{ticker}')
```

```
>>> plt.ylabel(ylabel='로그 수익률')
```

```
>>> plt.title(label=f'{ticker} 로그 수익률 상자 그림', fontweight='bold')
```

```
>>> plt.show()
```

[참고] 상자 밖 빨간 점은 이상치(극단값)를 의미하며, 상자(IQR) 구간은 평상시 변동 범위를 나타냅니다.
상자 그림을 시각화하면 히스토그램보다 이상치 파악이 쉽습니다.

로그 수익률 Normal Q-Q Plot 시각화

- 필요한 모듈을 임포트합니다.

```
>>> from scipy import stats
```

- 로그 수익률을 Normal Q-Q Plot으로 시각화합니다. *# [참고] 모든 점이 빨간색 직선 위에 놓여 있으면 데이터가 정규분포한다고 판단합니다.*

```
>>> stats.probplot(x=df['Log_Return'], dist='norm', plot=plt)
```

```
>>> plt.xlabel(xlabel='정규분포 기준 이론적 분위수')
```

```
>>> plt.ylabel(ylabel='표본 데이터 분위수')
```

```
>>> plt.title(label=f'{ticker} Normal Q-Q Plot', fontweight='bold')
```

```
>>> plt.show()
```

이동 변동성 선 그래프 시각화

- 이동 변동성을 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Volatility_20', color='red', lw=1)
```

```
>>> plt.axhline(y=df['Volatility_20'].mean(), color='0.5', ls='--', lw=0.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='표준편차')
```

```
>>> plt.title(label=f'{ticker} 20일 이동 표준편차', fontweight='bold')
```

```
>>> plt.show()
```

이동 평균 선 그래프 시각화

- 이동 평균을 선 그래프로 시각화합니다.

```
>>> sns.lineplot(data=df, x=df.index, y='Close', color='0.8', lw=1)
```

```
>>> sns.lineplot(data=df, x=df.index, y='Moving_Avg_20', color='blue', lw=1)
```

```
>>> sns.lineplot(data=df, x=df.index, y='Moving_Avg_60', color='red', lw=1.5)
```

```
>>> plt.xlabel(xlabel='거래일')
```

```
>>> plt.ylabel(ylabel='가격($)')
```

```
>>> plt.title(label=f'{ticker} 종가 및 이동 평균', fontweight='bold')
```

```
>>> plt.show()
```

종목 간 성과 비교

- 지금까지 주식 데이터를 수집하고, 전처리 및 분석 지표를 계산하여 시각화까지 일련의 분석 과정을 실습했습니다.
- 미국 주식과 국내 주식 종목을 참고하여 같은 기간 가장 높은 누적 수익률을 기록한 종목을 탐색하는 실습을 진행하겠습니다.
 - 같은 기간 동안 각 종목의 누적 수익률을 계산하고, 상위 3개 종목을 선택하세요.
 - 단, 전체 종목에서 변동성 상위 30%에 해당하는 종목은 제외합니다.

미국 주식 티커	국내 주식 티커
'AAPL', 'MSFT', 'NVDA', 'GOOGL', 'META', 'TSLA', 'AMZN', 'NFLX', 'AMD', 'JPM', 'BA', 'XOM', 'V',	'005930.KS', '000660.KS', '035420.KS', '035720.KS', '373220.KS', '051910.KS', '005380.KS', '000270.KS', '207940.KS', '068270.KS', '105560.KS', '055550.KS',

국내 주식 데이터 수집

KOSPI Stock Price Crawling

pykrx 라이브러리 소개

- pykrx는 한국거래소에서 제공하는 데이터를 기반으로 국내 주식(KOSPI, KOSDAQ, KONEX) 가격, 거래량, 투자자별 매매, 공모도 등의 정보를 수집할 수 있는 Python 라이브러리입니다.
 - 별도의 인증키 없이 한국 주식 시장 데이터를 데이터프레임 형태로 불러올 수 있습니다.
 - 국내 주식 시장 데이터에 최적화되어 있으며, 투자자별 매매동향(개인, 외국인, 기관), 공모도 잔고, 지수 데이터 등 yfinance에서 제공되지 않는 데이터를 수집할 수 있습니다.
 - 웹 크롤링 기반으로 제공되는 서비스이므로 웹 데이터 제공 구조가 변경될 경우 코드가 동작하지 않을 수 있습니다.
 - 재무제표, 펀더멘털 데이터의 지원이 제한적이므로 OPEN DART와 같은 공식 데이터 소스와 병행하는 것이 필요합니다.

필요한 모듈 импорт

- Colab에서 필요한 라이브러리 설치 여부를 확인합니다.

```
>>> !pip list | grep pykrx
```

- 필요한 라이브러리를 설치합니다.

```
>>> !pip install pykrx
```

[참고] 라이브러리를 설치하면서 의존성 라이브러리의 버전이 달라지면 Colab을 자동으로 재부팅합니다.
런타임 연결 해제 전까지 새로 설치한 라이브러리를 사용할 수 있습니다.

KOSPI 시가총액 조회 화면

- 네이버 증권 > 국내증시 > 시가총액을 클릭하면 KOSPI 상위 50위 종목을 확인할 수 있습니다.

증권 > 국내증시 > 시가총액

시가총액

항목을 자유롭게 변경하실 수 있습니다. | 최대 6개까지 설정 가능합니다.

☒ 거래량
 ☐ 매수호가
 ☐ 거래대금(백만)
 ☒ 시가총액(억)
 ☐ 영업이익(억)
 ☒ PER(배)

☐ 시가
 ☐ 매도호가
 ☐ 당일거래량
 ☐ 영업이익률(%)
 ☒ ROA(%)

☐ 고가
 ☐ 매수종가
 ☒ 외국인비율
 ☐ 부채총계(억)
 ☐ 당기순이익(억)
 ☐ ROE(%)

☐ 저가
 ☐ 매도종가
 ☒ 상장주식수(천주)
 ☐ 매출액(억)
 ☐ 주당순이익(원)
 ☐ PBR(배)

☐
☐ 매출증가율
 ☐
☐ 보통주배당금(원)
 ☐ 유보율(%)

전용하기 초기 항목으로

N	종목명	현재가	전일비	등락률	액면가	시가총액	상장주식수	외국인비율	거래량	PER	ROE	토론
1	삼성전자	204,250	▼ 6,250	-2.97%	100	12,090,860	5,919,638	48.46	13,528,117	31.12	10.85	
2	SK하이닉스	1,005,000	▼ 28,000	-2.71%	5,000	7,162,659	712,702	52.90	1,795,959	17.05	44.15	
3	삼성전자우	138,400	▼ 1,200	-0.86%	100	1,129,309	815,975	76.22	1,956,040	21.08	N/A	
4	현대차	501,000	▼ 7,000	-1.38%	5,000	1,025,836	204,758	27.83	386,386	14.18	8.41	
5	LG에너지솔루션	421,500	▲ 15,500	+3.82%	500	986,310	234,000	5.04	361,704	-91.93	-5.19	

https://finance.naver.com/item/main.naver?code=005930

[참고] 종목명에 마우스를 올려놓으면 여섯 자리 종목 코드(티커)가 포함된 링크를 확인할 수 있습니다.

<https://finance.naver.com/item/main.naver?code=005930>

HTTP 통신 리소스 찾기

- 타겟 페이지로 이동하여 수집할 데이터를 확인하고 크롬 개발자 도구를 엽니다.
 - 타겟 페이지: https://finance.naver.com/sise/sise_market_sum.naver

The screenshot shows the Chrome DevTools Network tab. The 'Network' tab is selected. The filter bar shows 'All' selected, with other filters like 'Fetch/XHR', 'Doc', 'CSS', 'JS', 'Font', 'Img', 'Media', 'Manifest', 'Socket', 'Wasm', and 'Other'. A red dashed box highlights the filter bar and the first row of the resource table.

All: 모든 리소스를 포함하고 있습니다.
 Doc: 정적인 HTML 리소스를 여기에서 찾습니다.
 Fetch/XHR: JavaScript 리소스를 여기에서 찾습니다.

Name	Status	Type	Initiator	Size	Time
# Name: 리소스 목록을 확인합니다. Status: 리소스의 응답 상태 코드를 확인합니다. Type: 리소스의 종류를 확인합니다.					

HTTP 통신 리소스 찾기(계속)

- 삭제 버튼 클릭하고 새로고침하면 HTTP 요청을 실행하고 리소스를 표시합니다.

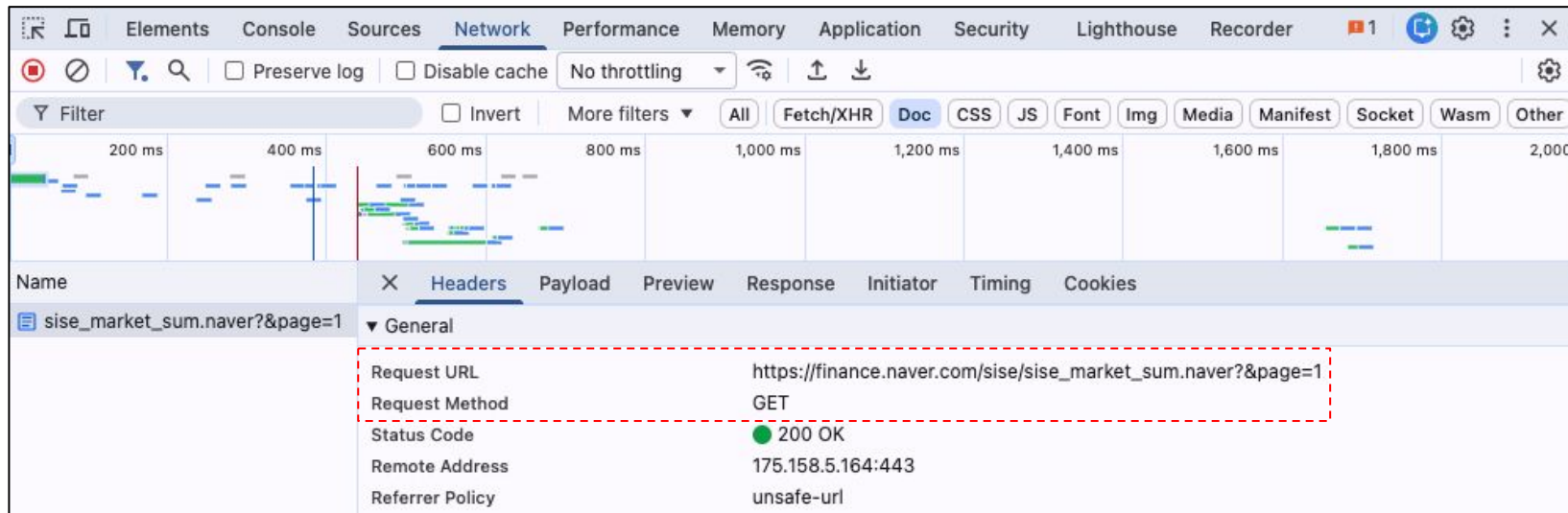
The screenshot shows the Chrome DevTools Network tab. The 'Delete' button (trash icon) is circled in red. The 'Doc' filter is also circled in red. The 'Preview' tab is selected and circled in red. The selected request is 'sise_market_sum.naver?&page=1'. The preview shows a table of stock market data.

N	종목명	현재가	전일비	등락률	액면가	시가총액	상장주식 수	외국인 비율	거래량	PER	ROE	토론
1	삼성전자	203,750	하락 6,750	-3.21%	100	12,061,262	5,919,638	48.46	15,190,039	31.04	10.85	토

[참고] 'sise'로 시작하는 리소스를 클릭하고 Preview로 이동하면, 해당 리소스의 HTTP 응답 바디 문자열(HTML)을 웹 페이지에 표시한 결과를 미리보기할 수 있습니다.

HTTP 통신 리소스 찾기(계속)

- 리소스의 Headers로 이동하여 요청 방식과 요청 URL을 확인합니다.



[주의] '&page=1'이 보이지 않는 경우, 웹 페이지 하단의 페이지 네비게이션을 클릭하면 요청 URL에 쿼리 문자열이 추가됩니다. 쿼리 문자열까지 확인해야 여러 페이지를 순회하며 데이터를 모두 수집할 수 있습니다.

HTTP 요청 실행

- 필요한 모듈을 임포트합니다.

```
>>> import requests
```

- 요청 URL을 설정합니다.

```
>>> url = 'https://finance.naver.com/sise/sise_market_sum.naver'
```

- 쿼리 문자열을 딕셔너리로 생성합니다.

```
>>> query = {'page': 1}
```

- HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url, params=query)
```

HTTP 응답 확인

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code # [참고] 상태 코드는 200입니다.
```

- HTTP 응답 헤더에서 콘텐츠 형태를 확인합니다.

```
>>> res.headers['content-type'] # [참고] 콘텐츠 형태는 'text/html; charset=EUC-KR'입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text)
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url)
```

HTML 데이터 처리

- 필요한 모듈을 임포트합니다.

```
>>> from bs4 import BeautifulSoup as BTS
```

- HTTP 응답 바디 문자열을 bs4.BeautifulSoup으로 변환하고 soup에 할당합니다.

```
>>> soup = BTS(markup=res.text, features='html.parser')
```

- soup을 확인합니다.

```
>>> soup
```

- HTML의 계층 구조(들여쓰기)를 반영한 문자열을 출력합니다.

```
>>> print(soup.prettify())
```

HTML 요소 찾기

- 선택 버튼을 한 번 클릭하고(파란색) 웹 페이지에서 수집할 텍스트를 선택합니다.

웹 페이지에서 마우스가 가리키는 HTML 요소를 파란색 블록으로 표시합니다.
[참고] 마우스를 클릭하면 해당 블록을 고정시킵니다.

크롬 개발자 도구에서 파란색 블록에 해당하는 HTML 요소를 회색 블록으로 표시하며
왼쪽 □를 클릭하면 □로 바뀌면서 해당 HTML 요소의 자식 요소를 펼쳐서 표시합니다.

HTML 요소의 CSS 선택자 복사

- 다음 과정을 거쳐 HTML 요소의 CSS 선택자를 복사합니다.

The screenshot shows the Chrome DevTools 'Elements' panel. It displays an HTML table with various attributes and nested elements like <caption>, <thead>, <tbody>, <tr>, <td>, and <a>. A right-click context menu is open over a table row. The menu options include 'Add attribute', 'Edit as HTML', 'Duplicate element', 'Delete element', 'Cut', 'Copy', 'Paste', and 'Hide element'. The 'Copy' option is expanded, showing sub-options: 'Copy element', 'Copy outerHTML', and 'Copy selector'. The 'Copy selector' option is highlighted. Red annotations are present: a circle with the number '1' points to the table row being right-clicked, and a circle with the number '2' points to the 'Copy selector' option in the context menu.

- ① HTML 요소 위에 마우스 오른쪽 버튼을 클릭합니다.
- ② 팝업 메뉴에서 Copy → Copy selector를 선택합니다.
- ③ Jupyter Notebook에 복사한 값을 붙여넣습니다.

종목명과 링크 수집

- 종목명을 포함하는 HTML 요소를 모두 선택하고 items1에 할당합니다.

```
>>> items1 = soup.select(selector='table.type_2 > tbody > tr > td > a.title')
```

- items1의 원소 개수를 확인합니다.

```
>>> len(items1)
```

- items1의 첫 번째 원소에서 종목명을 반환합니다.

```
>>> items1[0].text # [참고] 시작 태그와 종료 태그 사이의 문자열만 추출합니다.
```

- items1의 첫 번째 원소에서 종목명 링크를 반환합니다.

```
>>> items1[0]['href'] # [참고] href 속성에 연결된 링크만 추출합니다.  
                        Scheme과 Domain 없이 Path('/item/main.naver?code=005930') 부분만 있습니다.
```

PER 수집

- PER을 포함하는 HTML 요소를 모두 선택하고 items2에 할당합니다.

```
>>> items2 = soup.select(selector='table.type_2 > tbody > tr > td:nth-child(11)')
```

- items2의 원소 개수를 확인합니다.

```
>>> len(items2)
```

- items2의 첫 번째 원소에서 PER을 반환합니다.

```
>>> items2[0].text
```

[참고] KOSPI 상장 종목 중 ETF, ETN, 리츠 등 기업이 아닌 자산은 순이익 개념이 없어 PER을 계산할 수 없으므로 결측(N/A)으로 표시됩니다.

PER(주가수익비율)은 주가를 EPS(주당순이익)으로 나눈 값으로, 기업의 수익성을 기준으로 평가하는 지표입니다.

데이터프레임 생성

- 필요한 모듈을 임포트합니다.

```
>>> import pandas as pd
```

- 종목명, 링크 및 PER 열을 갖는 데이터프레임을 생성합니다.

```
>>> df_ticker = pd.DataFrame(data={'name': [item.text for item in items1],  
                                  'link': [item['href'] for item in items1],  
                                  'per': [item.text for item in items2]})
```

- df_ticker의 처음 5행을 확인합니다.

```
>>> df_ticker.head()
```

링크에서 티커 추출

- link의 각 원소 앞에 Scheme와 Domain을 추가합니다.

```
>>> df_ticker['link'] = 'https://finance.naver.com' + df_ticker['link']
```

- link의 각 원소에서 마지막 여섯 자리 숫자만 추출하여 ticker 열을 추가합니다.

```
>>> df_ticker['ticker'] = df_ticker['link'].str.extract(pat='([0-9]{6}$)')
```

- df_ticker의 처음 5행을 확인합니다.

```
>>> df_ticker.head()
```

- df_ticker의 정보를 확인합니다.

```
>>> df_ticker.info()
```

KOSPI 상장 종목 티커 수집 함수 생성

- KOSPI 티커를 데이터프레임으로 반환하는 함수를 생성합니다.(다음 페이지 코드 추가)

```
>>> def stock_ticker(page=1):  
  
    res = requests.get(url='https://finance.naver.com/sise/sise_market_sum.naver',  
                        params={'page': page})  
  
    soup = BeautifulSoup(markup=res.text, features='html.parser')  
  
    items1 = soup.select(selector='table.type_2 > tbody > tr > td > a.title')  
  
    items2 = soup.select(selector='table.type_2 > tbody > tr > td:nth-child(11)')
```

KOSPI 상장 종목 티커 수집 함수 생성(계속)

- 이전 페이지 코드에 이어서 다음 코드를 추가합니다.

```
df = pd.DataFrame(data={'name': [item.text for item in items1],  
                        'link': [item['href'] for item in items1],  
                        'per': [item.text for item in items2]})  
  
df['link'] = 'https://finance.naver.com' + df['link']  
  
df['ticker'] = df['link'].str.extract(pat='([0-9]{6}$)')  
  
return df
```

KOSPI 상장 종목 티커 수집

- 필요한 모듈을 임포트합니다.

```
>>> from tqdm.notebook import tqdm
```

```
>>> import time
```

- 반복문을 실행하여 KOSPI에 상장된 전 종목 티커를 수집합니다.

```
>>> for i in tqdm(range(1, 49)): # [참고] KOSPI 시가총액의 마지막 페이지를 확인하여 지정합니다.  
                                2페이지부터 49페이지까지 반복 실행합니다.
```

```
    df = stock_ticker(i + 1)
```

```
    df_ticker = pd.concat(objs=[df_ticker, df], ignore_index=True)
```

```
    time.sleep(1)
```


퀀트 투자 제외 종목

- 퀀트 투자 전략은 기업 가치를 일관된 기준으로 비교해야 합니다.
 - 유동성이 낮은 종목은 실제 투자 환경과 괴리가 발생할 수 있습니다.
 - 동일 기업이 여러 종목으로 포함되면 포트폴리오 비중이 왜곡될 수 있습니다.
 - 백테스트 과대평가를 방지하기 위해 종목 구성을 제한할 필요가 있습니다.
- 퀀트 투자 전략에서 분석 결과의 신뢰성과 해석 가능성을 확보를 위해 다음 종목을 제외합니다.
 - 일반주와 중복되어 가격 왜곡을 유발할 수 있는 우선주를 제외합니다.
 - 기업의 본질적 가치와 직접적인 연결이 없는 파생상품도 제외합니다.
 - 재무 지표가 없거나 계산이 어려운 종목은 객관적인 비교가 불가능하므로 제외합니다.

우선주 및 파생상품 제외

- df_ticker에서 ticker의 끝자리가 0이면 True, 0이 아니면 False인 cond1을 생성합니다.

```
>>> cond1 = df_ticker['ticker'].str.extract(pat='([0-6]$)')[0].eq(other='0')
```

- df_ticker에서 per이 'N/A'가 아니면 True, 'N/A'이면 False인 cond2를 생성합니다.

```
>>> cond2 = df_ticker['per'].ne(other='N/A')
```

- df_ticker에서 cond1과 cond2 모두 True인 행을 선택하고 df_ticker에 재할당합니다.

```
>>> df_ticker = df_ticker.loc[cond1 & cond2, :]
```

- df_ticker의 인덱스를 초기화합니다.

```
>>> df_ticker = df_ticker.reset_index(drop=True)
```

KOSPI 상장 종목 티커 확인

- df_ticker의 행 개수와 열 개수를 확인합니다.

```
>>> df_ticker.shape # [참고] 데이터프레임의 shape 속성은 행 개수와 열 개수를 튜플로 반환합니다.
```

- df_ticker의 처음 5행을 확인합니다.

```
>>> df_ticker.head()
```

- df_ticker의 마지막 5행을 확인합니다.

```
>>> df_ticker.tail()
```

- df_ticker의 정보를 확인합니다.

```
>>> df_ticker.info()
```

첫 번째 티커의 주식 데이터 수집

- 필요한 모듈을 임포트합니다.

```
>>> from pykrx import stock
```

- 주식 데이터 수집 시작일자를 설정합니다.

```
>>> start_date = '20191230' # [참고] 시작일자와 종료일자를 'YYYYMMDD' 형태의 문자열로 설정해야 합니다.  
                           국내 주식 시장은 매년 12월 31일에 휴장합니다.
```

- 주식 데이터 수집 종료일자를 설정합니다.

```
>>> end_date = pd.Timestamp.today().strftime(format='%Y%m%d')
```

- 첫 번째 종목명과 티커를 설정합니다.

```
>>> name, ticker = df_ticker.loc[:, ['name', 'ticker']].iloc[0]
```

첫 번째 티커의 주식 데이터 수집(계속)

- 첫 번째 티커의 주식 데이터를 수집합니다.

```
>>> df_price = stock.get_market_ohlcw_by_date(fromdate=start_date,  
                                              todate=end_date, # [참고] todate 매개변수에  
                                              # 지정된 일자를 포함합니다.  
                                              ticker=ticker)
```

- 종목명을 df_price의 첫 번째 열에 삽입합니다.

```
>>> df_price.insert(loc=0, column='종목명', value=name)
```

- 티커를 df_price의 두 번째 열에 삽입합니다.

```
>>> df_price.insert(loc=1, column='티커', value=ticker)
```

첫 번째 티커의 주식 데이터 확인

- df_price의 처음 5행을 확인합니다.

```
>>> df_price.head()
```

- df_price의 마지막 5행을 확인합니다.

```
>>> df_price.tail()
```

- df_price의 정보를 확인합니다.

```
>>> df_price.info()
```

- df_price 날짜 인덱스의 최솟값과 최댓값을 확인합니다.

```
>>> df_price.index.min(), df_price.index.max()
```

KOSPI 전 종목 주식 데이터 수집

- 반복 실행할 횟수를 설정합니다.

```
>>> n = df_ticker.shape[0]
```

- 반복문을 실행하여 KOSPI에 상장된 전 종목 주식 데이터를 수집합니다.(다음 페이지 코드 추가)

```
>>> for i in tqdm(range(1, n)):
```

```
    name, ticker = df_ticker.loc[:, ['name', 'ticker']].iloc[i]
```

```
    df = stock.get_market_ohlcw_by_date(fromdate=start_date,
```

```
                                       todate=end_date,
```

```
                                       ticker=ticker)
```

KOSPI 전 종목 주식 데이터 수집(계속)

- 이전 페이지 코드에 이어서 다음 코드를 추가합니다.

```
df.insert(loc=0, column='종목명', value=name)
```

```
df.insert(loc=1, column='티커', value=ticker)
```

```
if df.shape[0] >= 1:
```

```
df_price = pd.concat(objs=[df_price, df]) # [주의] df의 인덱스는 날짜 인덱스이므로  
                                           기존 인덱스를 초기화하면 안됩니다.
```

[참고] 로컬 컴퓨터에서 실행하는 것이 Colab에서 실행하는 것보다 5~6배 빠릅니다.
Colab은 구글 클라우드의 해외 리전에서 실행되는 경우가 많으므로 요청 1번당 왕복 지연시간이 커질 수 있습니다.

- df_price의 정보를 확인합니다.

```
>>> df_price.info()
```


구글 드라이브 마운트

- 필요한 모듈을 임포트합니다.

```
>>> from google.colab import drive
```

```
>>> import os
```

- 구글 드라이브에 연결합니다.

```
>>> drive.mount(mountpoint='/content/drive')
```

- 구글 드라이브로 작업 경로를 변경합니다.

```
>>> os.chdir(path='/content/drive/MyDrive/Quant/data')
```

외부 파일로 저장

- `df_ticker`를 csv 파일로 저장합니다.

```
>>> df_ticker.to_csv(path_or_buf='KOSPI_Stock_Ticker.csv', index=False)
```

- `df_price`를 csv 파일로 저장합니다.

```
>>> df_price.to_csv(path_or_buf='KOSPI_Stock_Price.csv')
```

[주의] df_price는 날짜 인덱스를 가지고 있으므로 csv 파일로 저장할 때 인덱스를 추가해야 합니다.

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir()
```

퀀트 투자 전략 백테스트

Quant Strategy Backtest

주가와 거래량만으로 시작할 수 있는 투자 전략

- 주가 데이터를 활용하여 다음 세 가지 핵심 질문에 대한 답을 탐색하는 과정입니다.

질문	핵심 지표	상세 내용
무엇을 살 것인가? (유동성과 추세)	거래대금 이동 평균선	- 시장의 확신이 실린 종목인가? - 현재 주가는 상향 추세인가, 하향 추세인가?
언제 살 것인가? (진입과 청산)	RSI 볼린저 밴드	- 시장이 심리적으로 과하게 쏠린 지점은 어디인가? - 현재 주가가 통계적인 범위를 벗어났는가?
어떻게 지킬 것인가? (위험 관리)	ATR	- 이 종목의 변동성에 맞는 손절선은 어디인가?

[참고] RSI는 Relative Strength Index(상대강도지수), ATR은 Average True Range(평균 실제 변동폭)입니다.

시장의 흐름과 신뢰도: 이동 평균선과 거래대금

- 이동 평균선은 일정 기간의 주가를 산술 평균하여 추세 방향을 알려주는 나침반 역할을 합니다.

$$MA = \frac{1}{n} \sum_{i=1}^n P_i$$

- 주가 > 단기(20일) > 장기(60일)의 정배열 상태일 때 상승 추세로 해석됩니다.
- 거래대금은 당일 거래된 주식 수에 가격을 곱한 총 금액입니다.

$$\text{거래대금} = \text{당일 종가} \times \text{당일 거래량}$$

- 시장 참여자들이 해당 가격 변동을 얼마나 신뢰하는지 증명하는 확신의 척도가 됩니다.
 - 거래대금이 동반되지 않는 추세는 지속 가능성이 낮을 수 있으므로, 유동성이 풍부한 종목을 선별하는 필터로 거래대금을 활용하는 것이 좋습니다.

매수와 매도의 적기: RSI와 볼린저 밴드

- RSI^{Relative Strength Index}는 최근 주가의 상승폭과 하락폭을 비교하여 현재 시장의 에너지가 과열 또는 침체되었는지를 0~100 범위의 수치로 나타내는 강도 지표입니다.

$$RSI = 100 - \left(\frac{100}{1 + RS} \right), RS = \frac{n\text{일간 상승폭 평균}}{n\text{일간 하락폭 평균}} \quad \# [참고] n을 14로 설정합니다.$$

- RSI가 30 이하면 과매도(매수 고려), 70 이상이면 과매수(매도 준비)로 판단합니다.
- 볼린저 밴드^{Bollinger Bands}는 이동 평균선을 중심으로 주가가 움직이는 통계적 범위를 표준편차로 설정한 지표입니다.
[참고] 일반적으로 20일 이동 평균과 이동 표준편차를 사용하며, 승수 k는 2로 설정합니다.
정규분포를 가정할 때 평균에서 $\pm 2\sigma$ 범위 안에 값이 존재할 확률은 약 95.4%입니다.

$$\text{Lower Band} = MA(n) - (k \times \sigma), \text{Upper Band} = MA(n) + (k \times \sigma)$$

- 주가가 상단선에 닿으면 평균으로 회귀하려는 성질을 이용해 고점 매도하거나(평균 회귀), 상단선을 강력하게 뚫으면 새로운 추세가 시작된 것으로 판단하여 매수합니다(추세 돌파).

[참고] 볼린저 밴드의 스퀴즈 현상

- 볼린저 밴드에서 가장 중요한 순간은 밴드의 폭이 좁아지는 스퀴즈 현상을 보일 때입니다.
 - 상하단 폭이 좁아지면 조만간 큰 시세가 터질 것임을 예고합니다.
 - 밴드 폭이 좁아지는 구간은 변동성이 축소된 상태이며, 이후 변동성이 확대될 가능성이 높은 구간입니다. 다만 방향성을 예측하는 것은 불가능합니다.
 - 밴드 폭이 좁아지는 스퀴즈 현상을 보일 때, 한쪽 방향으로 뚫고 나가는 때가 추세 돌파 전략이 빛을 발하는 시점입니다.
 - 스퀴즈 현상을 보일 때 평균 회귀 전략은 위험할 수 있습니다.

계좌를 지키는 안전장치: ATR

- ATR^{Average True Range}은 현재 주가가 하루에 평균적으로 움직이는 범위를 수치화한 것입니다.

$$TR = \max(\text{High} - \text{Low}, |\text{High} - \text{Pre Close}|, |\text{Low} - \text{Pre Close}|)$$

$$ATR = \frac{1}{n} \sum_{i=1}^n TR_i$$

- ATR은 종목마다 최적의 손절선을 설정하게 함으로써 계좌의 급격한 손실을 방어하는 데 유용한 수비적인 지표입니다.
- 손절선을 현재 주가에서 ATR의 승수 k만큼 아래로 잡습니다.
 - k를 1.5~2.0으로 설정하면 공격적인 단기 매매, 2.5~3.0으로 설정하면 중장기 추세 매매입니다.
 - 변동성이 큰 종목에서는 손절선을 넓게, 변동성이 작은 종목에서는 손절선을 좁게 잡게 됩니다.

다섯 가지 지표를 결합한 통합 투자 전략

- 다섯 가지 지표를 하나의 논리적인 흐름으로 결합하여, 유동성 검증부터 위험 관리까지 포함된 완성된 투자 레시피를 구성합니다.

구분	핵심 지표	상세 내용
유니버스 필터링	거래대금	최근 20일 평균 거래대금이 코스피 상위 20% 이내인 종목만 선정하여 언제든지 사고팔 수 있는 유동성을 확보합니다.
추세 확인	이동 평균선	이동 평균선이 정배열(20일선 > 60일선) 상태인 종목을 선택합니다. 이는 중장기 상승 추세를 의미합니다.
진입 타이밍	RSI 볼린저 밴드	상승 추세 종목 중 주가가 볼린저 밴드 하단선에 근접하고, RSI가 낮아졌을 때 기술적 반등을 기대하고 매수합니다.
리스크 관리	ATR	매수 직후에 현재 주가에서 ATR의 2.5배만큼 아래 지점으로 손절선을 설정합니다.

백테스트의 필요성

- 퀀트 투자에서 백테스트는 단순한 시뮬레이션이 아니라, 소중한 자산을 투입하기 전 통과해야 하는 최종 안전 점검입니다. # [주의] 과거의 수익이 미래의 수익을 보장하지 않습니다.
 - 과거 데이터를 통해 전략의 승률과 수익성을 숫자로 확인함으로써 하락장에서 흔들리지 않고 전략을 유지할 수 있는 심리적 근거가 됩니다.
 - 과거 폭락장에서 이 전략의 최대 낙폭이 어느 정도인지 미리 확인하여 감당할 수 있는 위험 수준인지 사전에 파악할 수 있습니다.
 - 특정 종목 또는 특정 시기에 우연히 잘 맞았던 전략은 아닌지 검증함으로써 다양한 시장 상황에서 유효한 전략인지 확인할 수 있습니다.
 - 시장 지수 대비 초과 수익을 낼 수 있는지 확인할 수 있습니다.

백테스트 평가 지표

평가 지표	계산식	상세 내용
CAGR (연평균 성장률)	$CAGR = \left(\frac{\text{최종 자산}}{\text{초기 자산}} \right)^{1/n} - 1$	- 특정 기간 수익률을 연단위로 환산한 복리 수익률입니다. - 서로 다른 기간의 전략들을 비교할 수 있게 해줍니다. - CAGR은 Compound Annual Growth Rate의 줄임말입니다.
MDD (최대 낙폭)	$MDD = \frac{\text{전저점} - \text{전고점}}{\text{전고점}}$	- 특정 기간 내 발생한 전고점 대비 최대 손실입니다. - 투자자가 실전에서 마주할 공포의 크기를 의미합니다. - 전략의 생존 가능성을 판단하는 척도입니다.
Sharpe Ratio (샤프 지수)	$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p}$	- 초과 수익률을 변동성으로 나누어 계산한 값입니다. - 무위험 수익률은 0 또는 국고채 금리로 가정합니다. - 얼마나 가성비 있게 벌었는가를 나타냅니다.

3단계 백테스트 전략 로드맵

구분	전략	상세 내용
1단계 (공격형)	단순 추세 추종	- 유동성 스크리닝 + 정배열 종목 즉시 매수 + 고정 -5% 손절, 10% 익절 - 상승 추세가 있는 종목을 매수하면 수익을 얻을 수 있을까?
2단계 (방어형)	진입 타이밍 최적화	- 1단계 전략 + RSI & 볼린저 밴드 눌림목 진입 + 고정 -5% 손절, 10% 익절 - 아무 때나 매수하는 것보다 눌림목에 매수하는 것이 유리할까?
3단계 (균형형)	동적 리스크 관리	- 2단계 전략 + ATR을 활용한 트레일링 스탑 익절/손절선 설정 - 종목별 보폭에 맞춘 손절이 최대 낙폭을 줄일 수 있을까?

[참고] 고정 -5% 손절 기준으로는 시장의 보폭을 반영하지 못합니다. 3단계에서 ATR의 2.5배를 기준으로 주가가 오르면 손절선도 함께 높이는 트레일링 스탑을 적용함으로써 하락장에서는 빠르게 최대 낙폭을 방어하고, 상승장에서는 수익을 끝까지 추적하여 전략의 가성비(샤프 지수)를 극대화합니다.

3단계 백테스트 결과에서 1~2단계보다 수익률이 낮을 수 있지만 최대 낙폭을 크게 줄일 수 있습니다.(Chandelier Exit)

[참고] 백테스트의 함정: 생존 편향과 룯어헤드 편향

구분	상세 내용
생존 편향	- 정의: 과거 상장 폐지된 종목을 제외하고 현재 살아남은 종목들로 과거를 시뮬레이션할 때 발생하는 오류입니다. - 문제점: 상장 폐지 종목이 제외되므로, 전략의 평균 수익률은 실제보다 높고, 최대 낙폭은 실제보다 낮게 왜곡됩니다. - 해결책: 상장 폐지된 종목을 포함하거나, 백테스트 결과에서 일정 비율의 생존 프리미엄을 차감하여 보수적으로 해석해야 합니다.
룰어헤드 편향	- 정의: 상장 폐지 여부 및 확정 실적 발표 등 미래에 일어날 일을 과거 시점에서 미리 알고 투자하는 것처럼 계산하는 오류입니다. - 문제점: 어차피 우량주가 될 종목을 과거에 샀다고 가정하는 것이므로, 현실에서 불가능한 비정상적인 수익률이 도출될 수 있습니다. - 해결책: 각 시점마다 그 당시에서 알 수 있었던 정보만을 사용하여 매매 결정을 내리도록 통제해야 합니다.

백테스트 관련 주의사항

- 백테스트를 실행할 때 주의해야 할 내용은 다음과 같습니다.
 - 현재 살아 남은 종목만 사용하면 결과가 왜곡되는 생존 편향을 주의해야 합니다.
 - 미래 데이터를 사용하면 결과가 왜곡되는 룩어헤드 편향을 주의해야 합니다.
 - 당일 종가로 지표를 계산했다면, 다음 거래일에 체결하는 것으로 가정해야 합니다.
 - 종가 기준 매수/매도는 현실과 차이가 있습니다.(슬리피지 & 호가 차이)
 - 슬리피지는 주문 가격과 실제 체결 가격 사이에 발생하는 차이를 의미합니다.
 - 거래 수수료와 세금을 반영하지 않으면 수익률이 과대평가됩니다.
 - 초기 자본에 크기에 따라 백테스트 결과가 달라질 수 있습니다.

구글 드라이브 마운트

- 필요한 모듈을 임포트합니다.

```
>>> from google.colab import drive
```

```
>>> import os
```

```
>>> import pandas as pd
```

- 구글 드라이브에 연결합니다.

```
>>> drive.mount(mountpoint='/content/drive')
```

- 구글 드라이브로 작업 경로를 변경합니다.

```
>>> os.chdir(path='/content/drive/MyDrive/Quant/data')
```

실습 데이터셋 준비

- 주가 데이터를 읽고 df_price에 할당합니다.

```
>>> df_price = pd.read_csv(filepath_or_buffer='KOSPI_Stock_Price.csv',  
                             dtype={'티커': object})
```

- df_price의 처음 5행을 확인합니다.

```
>>> df_price.head()
```

- df_price의 정보를 확인합니다.

```
>>> df_price.info()
```


데이터 전처리

- df_price에서 날짜를 날짜시간형으로 변환합니다.

```
>>> df_price['날짜'] = pd.to_datetime(arg=df_price['날짜'])
```

- df_price를 티커와 날짜 기준으로 오름차순 정렬합니다.

```
>>> df_price = df_price.sort_values(by=['티커', '날짜'])
```

- df_price에서 날짜를 인덱스로 설정합니다.

```
>>> df_price = df_price.set_index(keys='날짜')
```

- df_price의 처음 5행을 확인합니다.

```
>>> df_price.head()
```

파생변수 생성: 거래대금 & 이동 평균

- 거래대금을 계산합니다.

```
>>> df_price['거래대금'] = df_price['종가'] * df_price['거래량']
```

- 단기(20일) 이동 평균을 계산합니다.

```
>>> ma_20 = lambda x: x.rolling(window=20).mean()
```

```
>>> df_price['MA_20'] = df_price.groupby(by='티커')['종가'].transform(func=ma_20)
```

- 장기(60일) 이동 평균을 계산합니다.

```
>>> ma_60 = lambda x: x.rolling(window=60).mean()
```

```
>>> df_price['MA_60'] = df_price.groupby(by='티커')['종가'].transform(func=ma_60)
```

파생변수 생성: 거래대금 & 이동 평균(계속)

- 전일 종가를 추가합니다.

```
>>> df_price['전일종가'] = df_price.groupby(by='티커')['종가'].shift(periods=1)
```

- 전일 거래대금을 추가합니다.

```
>>> df_price['전일거래대금'] = df_price.groupby(by='티커')['거래대금'].shift(periods=1)
```

- df_price의 처음 5행을 확인합니다.

```
>>> df_price.head()
```

작업 경로 확인 및 변경

- 현재 작업 경로를 확인합니다.

```
>>> os.getcwd()
```

- 구글 드라이브로 작업 경로를 변경합니다.

```
>>> os.chdir(path='../code')
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> sorted(os.listdir())
```

[참고] 퀀트 투자 전략 백테스트 전용 클래스를 두 개의 py 파일(모듈)로 제공합니다.
code 폴더에 'quant_strategy.py'와 'quant_backtest.py' 파일을 미리 저장해야 이후 실습을 진행할 수 있습니다.

퀀트 투자 전략 1단계 백테스트

- 필요한 모듈을 임포트합니다.

```
>>> from quant_strategy import Step1Strategy
```

```
>>> from quant_backtest import Backtester
```

- 1단계 전략을 생성합니다.

```
>>> strategy_1 = Step1Strategy(top_n=10, max_n=5, stop_loss=-0.05, take_profit=0.10)
```

*# [참고] top_n과 max_n 매개변수에 매수 후보 종목 개수(10개)와 최대 포지션 개수(5개)를 설정합니다.
stop_loss와 take_profit 매개변수에 손절(-5%)과 익절(10%) 기준을 설정합니다.*

- 1단계 전략의 백테스트 객체를 생성합니다.

```
>>> backtest_1 = Backtester(strategy=strategy_1, init_cash=10_000_000)
```

퀀트 투자 전략 1단계 백테스트(계속)

- 1단계 전략의 백테스트를 수행합니다.

```
>>> result_1 = backtest_1.run_backtest(df=df_price)
```

- result_1의 마지막 5행을 확인합니다.

```
>>> result_1.tail()
```

- 1단계 전략의 백테스트 결과를 선 그래프로 시각화합니다.

```
>>> backtest_1.plot(result=result_1)
```

- 1단계 전략의 백테스트 성능 지표를 확인합니다.

```
>>> backtest_1.evaluate(result=result_1) # [참고] 연복리 수익률이 클수록, 최대 낙폭이 작을수록,  
                                           샤프 지수가 클수록 좋은 투자 전략이라고 판단합니다.
```

Colab에서 한글 폰트 설치

- 현재 Colab에 설치된 폰트 경로 목록을 확인합니다.

```
>>> !fc-list # [참고] fc-list는 리눅스 fontcoding 계열 명령어로, 현재 설치된 폰트 경로 목록을 출력합니다.
```

- 불필요한 경로를 제거하고 전체 폰트 이름만 출력합니다.

```
>>> !fc-list : family # [참고] 콜론은 fontcoding의 쿼리 구문 시작 기호입니다.  
콜론 오른쪽에 조건을 추가하지 않았으므로 경로 없이 전체 폰트 이름만 출력합니다.
```

- Colab에서 사용할 나눔 폰트를 설치합니다.

```
>>> !apt-get install -y fonts-nanum # [참고] apt-get은 리눅스 저장소에 등록된 패키지를 의존성까지  
포함하여 설치/업데이트/삭제하는 명령어입니다.
```

- 불필요한 경로를 제거하고 한글 폰트 이름만 중복 없이 오름차순 정렬하여 출력합니다.

```
>>> !fc-list :lang=ko family | sort | uniq # [참고] 콜론 오른쪽에 한글 폰트만 필터링하는 조건과  
중복 제거 및 오름차순 정렬 옵션을 추가합니다.
```

한글 폰트명 탐색

- 필요한 모듈을 임포트합니다.

```
>>> import matplotlib.font_manager as fm # [참고] 컴퓨터에 설치된 폰트 정보를 확인할 때 사용합니다.
```

```
>>> import matplotlib.pyplot as plt
```

```
>>> import seaborn as sns
```

- 현재 사용 중인 컴퓨터에 설치된 전체 폰트 파일명을 리스트로 생성합니다.

```
>>> fontList = fm.findSystemFonts(fonttext='ttf')
```

- fontList의 원소 개수를 확인합니다.

```
>>> len(fontList)
```


한글 폰트명 탐색(계속)

- fontList에서 특정 폰트명을 포함하는 파일명을 선택하여 fontPath에 할당합니다.

```
>>> fontPath = sorted([font for font in fontList if 'Nanum' in font])
```

- fontPath를 확인합니다.

```
>>> fontPath
```

- 반복문으로 컴퓨터에 설치된 폰트명을 리스트로 반환합니다.

```
>>> [fm.FontProperties(fname=font).get_name() for font in fontPath]
```

- matplotlib 라이브러리의 폰트 관리자 객체를 초기화합니다.

```
>>> fm.fontManager.__init__() # [참고] 컴퓨터에 설치된 폰트 파일들을 다시 스캔하여 내부 폰트 캐시를  
                             재구성하여 새로 설치한 한글 폰트를 사용할 수 있게 합니다.
```

그래프 요소 설정

- 한글 폰트와 글자 크기를 설정합니다. # [참고] rc는 runtime configuration를 의미하며 pyplot을 실행하는 환경을 의미합니다.

```
>>> plt.rc(group='font', family='NanumBarunGothic', size=10)
```

- 그래프 크기와 해상도를 설정합니다.

```
>>> plt.rc(group='figure', figsize=(12, 4), dpi=120)
```

- 축에 유니코드 마이너스를 출력하지 않도록 설정합니다.

```
>>> plt.rc(group='axes', unicode_minus=False) # [참고] x축과 y축 눈금에서 음수 앞에 '-' 대신 '␣'를 출력하도록 않도록 설정합니다.
```

- 범례에 채우기 색과 테두리 색을 추가합니다. # [참고] fc 매개변수에 채우기 색을, ec 매개변수에 테두리 색을 지정합니다. '0'은 검정, '1'은 흰색입니다.

```
>>> plt.rc(group='legend', frameon=True, fc='0.9', ec='0.9')
```

최대 낙폭 시각화

- 필요한 모듈을 임포트합니다.

```
>>> from quant_visualize import plot_drawdown
```

- 1단계 백테스트 결과의 최대 낙폭을 시각화합니다.

```
>>> plot_drawdown(result=result_1, title='Step1 Drawdown')
```

파생변수 생성: RSI

- RSI를 계산하는 함수를 생성합니다.(다음 페이지 코드 추가)

```
>>> def calc_rsi(sr, window=14):  
  
    delta = sr.diff(periods=1)  
  
    gain = delta.clip(lower=0)  
  
    loss = -delta.clip(upper=0)  
  
    avg_gain = gain.rolling(window=window).mean()  
  
    avg_loss = loss.rolling(window=window).mean()
```

파생변수 생성: RSI(계속)

- 이전 페이지 코드에 이어서 다음 코드를 추가합니다.

```
rs = avg_gain / avg_loss  
  
rsi = 100 - (100 / (1 + rs))  
  
return rsi
```

- RSI를 계산합니다.

```
>>> df_price['RSI_14'] = df_price.groupby(by='티커')['종가'].transform(func=calc_rsi)
```

파생변수 생성: 볼린저 밴드

- 볼린저 밴드에서 사용할 종가의 20일 이동 표준편차를 계산합니다.

```
>>> ms_20 = lambda x: x.rolling(window=20).std()
```

```
>>> df_price['BB_STD_20'] = df_price.groupby(by='티커')['종가'].transform(func=ms_20)
```

- 볼린저 밴드 상단과 하단을 계산합니다.

```
>>> df_price['BB_Upper'] = df_price['MA_20'] + 2 * df_price['BB_STD_20']
```

```
>>> df_price['BB_Lower'] = df_price['MA_20'] - 2 * df_price['BB_STD_20']
```

볼린저 밴드 놀림목 시각화

- 필요한 모듈을 임포트합니다.

```
>>> from quant_visualize import plot_bollinger_signal
```

- 삼성전자 주가 데이터를 준비합니다.

```
>>> df_one = df_price.loc[df_price['티커'].eq(other='005930')]
```

- 볼린저 밴드 놀림목을 시각화합니다.

```
>>> plot_bollinger_signal(df=df_one, rsi_threshold=45, bu_buffer=1.1)
```

퀀트 투자 전략 2단계 백테스트

- 필요한 모듈을 임포트합니다.

```
>>> from quant_strategy import Step2Strategy
```

- 2단계 전략을 생성합니다.

```
>>> strategy_2 = Step2Strategy(top_n=10, max_n=5, stop_loss=-0.05, take_profit=0.10,  
                                rsi_threshold=45, bb_buffer=1.1)
```

[참고] 과매수 판단 기준,
볼린저 밴드 하단 여유폭
승수를 설정합니다.

- 2단계 전략의 백테스트 객체를 생성합니다.

```
>>> backtest_2 = Backtester(strategy=strategy_2, init_cash=10_000_000)
```


퀀트 투자 전략 2단계 백테스트(계속)

- 2단계 전략의 백테스트를 수행합니다.

```
>>> result_2 = backtest_2.run_backtest(df=df_price)
```

- result_2의 마지막 5행을 확인합니다.

```
>>> result_2.tail()
```

- 2단계 전략의 백테스트 결과를 선 그래프로 시각화합니다.

```
>>> backtest_2.plot(result=result_2)
```

- 2단계 전략의 백테스트 성능 지표를 확인합니다.

```
>>> backtest_2.evaluate(result=result_2)
```

파생변수 생성: ATR

- 행별로 TR(실제 변동폭)을 계산합니다.

```
>>> tr1 = df_price['고가'] - df_price['저가']
```

```
>>> tr2 = (df_price['고가'] - df_price['전일종가']).abs()
```

```
>>> tr3 = (df_price['저가'] - df_price['전일종가']).abs()
```

```
>>> df_price['TR'] = pd.concat(objs=[tr1, tr2, tr3], axis=1).max(axis=1)
```

- ATR을 계산합니다.

```
>>> ma_14 = lambda x: x.rolling(window=14).mean()
```

```
>>> df_price['ATR_14'] = df_price.groupby(by='티커')['TR'].transform(func=ma_14)
```

ATR 기반 손절선 변화 시각화

- 필요한 모듈을 임포트합니다.

```
>>> from quant_visualize import plot_atr_trainling_stop
```

- 삼성전자 주가 데이터를 준비합니다.

```
>>> df_one = df_price.loc[df_price['티커'].eq(other='005930')]
```

- 볼린저 밴드 놀림목을 시각화합니다.

```
>>> plot_atr_trainling_stop(df=df_one, atr_multiplier=2.5, entry_date='2026-01-01')
```

퀀트 투자 전략 3단계 백테스트

- 필요한 모듈을 임포트합니다.

```
>>> from quant_strategy import Step3Strategy
```

- 3단계 전략을 생성합니다.

```
>>> strategy_3 = Step3Strategy(top_n=10, max_n=5, rsi_threshold=45, bb_buffer=1.1,  
                                atr_multiplier=2.5) # [참고] 손절선 상향 조절 승수를 설정합니다.
```

- 3단계 전략의 백테스트 객체를 생성합니다.

```
>>> backtest_3 = Backtester(strategy=strategy_3, init_cash=10_000_000)
```

퀀트 투자 전략 3단계 백테스트(계속)

- 3단계 전략의 백테스트를 수행합니다.

```
>>> result_3 = backtest_3.run_backtest(df=df_price)
```

- result_3의 마지막 5행을 확인합니다.

```
>>> result_3.tail()
```

- 3단계 전략의 백테스트 결과를 선 그래프로 시각화합니다.

```
>>> backtest_3.plot(result=result_3)
```

- 3단계 전략의 백테스트 성능 지표를 확인합니다.

```
>>> backtest_3.evaluate(result=result_3)
```

단계별 백테스트 결과 비교

- 필요한 모듈을 임포트합니다.

```
>>> from quant_visualize import plot_backtest_compare
```

- 1~3단계별 백테스트 결과를 딕셔너리로 저장합니다.

```
>>> result_dict = {'Step1': result_1, 'Step2': result_2, 'Step3': result_3}
```

- 1~3단계별 백테스트 결과를 선 그래프로 시각화하여 비교합니다.

```
>>> plot_backtest_compare(result_dict=result_dict, init_cash=10_000_000)
```

퀀트 전략 vs 개별 종목 매수 & 보유 결과 비교

- 필요한 모듈을 임포트합니다.

```
>>> from quant_visualize import plot_strategy_vs_buy_hold
```

- 퀀트 전략과 개별 종목을 매수하고 보유한 결과를 비교합니다.

```
>>> plot_strategy_vs_buy_hold(result=result_3, df=df_one, init_cash=10_000_000)
```

End of Document